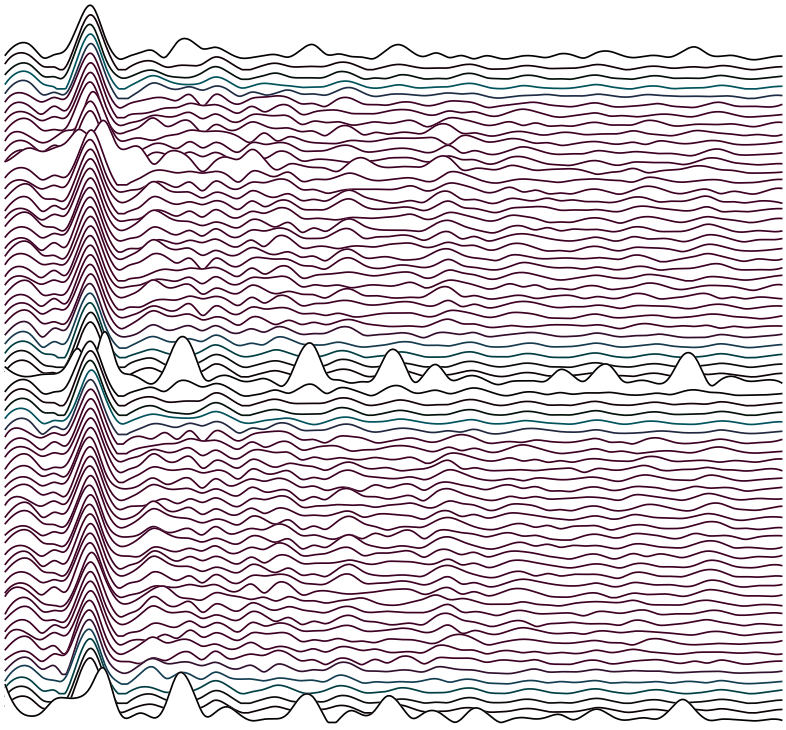
From Python to Silicon

A Technical Report on
Logistics, Power, Analog Synthesis, and Signal Analysis

Jackson Ferguson
University of Victoria
Astrophysics



Title graphic generated by the author (Python); source code in the accompanying repository:
https://github.com/JacksonFergusonDev/systems-audio-lab.

October 13, 2026

Abstract

This document records a recursive engineering journey: the design and fabrication of the tools required to build a specific device. Identifying a gap between software architecture proficiency and physical implementation, I undertook a project to build an analog guitar pedal (The Red Llama). However, the standard hobbyist workflow proved insufficient for rigorous engineering.

This technical report details the development of four interdependent systems: (1) **Star Ground**, a deterministic Python logistics engine for BOM management; (2) A thermal-managed **Linear Power Supply**; (3) The **Red Llama** analog overdrive circuit; and (4) A custom **RP2040 Data Acquisition System** (DAQ) used to spectrally validate the final build and to measure its swept frequency response and harmonic distortion.

Contents

1 Introduction **3**
- 1.1 The Systems Engineering Imperative . . . 3
- 1.2 Problem Formulation . . . 3
- 1.3 Methodology: The Recursive Solution . . . 4
- 1.4 Report Structure . . . 4

2 The Logistics Engine (Software) **5**
- 2.1 Problem Definition: Logistical Entropy . . . 5
- 2.2 System Architecture: Star Ground . . . 5
  - 2.2.1 Design Philosophy: Determinism over AI . . . 5
- 2.3 Algorithmic Logic: Nerd Economics . . . 6
- 2.4 The "Silicon Sommelier" . . . 6
- 2.5 Generated Artifacts . . . 6
- 2.6 Validation . . . 7

3 Infrastructure: Linear Power Supply **8**
- 3.1 Project Motivation . . . 8
- 3.2 Theory of Operation . . . 8
- 3.3 Thermal Management Strategy . . . 9
  - 3.3.1 The Solution . . . 9
- 3.4 Component Selection . . . 9

4 Device Under Test: Red Llama Overdrive **11**
- 4.1 Circuit Topology . . . 11
- 4.2 Procurement & Fabrication . . . 11
- 4.3 Engineering Deviation: The Diode Mod . . . 11
- 4.4 The Validation Gap . . . 12

5 Instrumentation: RP2040 DAQ **13**
- 5.1 System Architecture . . . 13
- 5.2 Analog Front End (AFE) Theory . . . 13
  - 5.2.1 Input Protection I . . . 13
  - 5.2.2 High-Pass Filter (AC Coupling) . . . 13
  - 5.2.3 Bias & Switched Reference Topology . . . 14
  - 5.2.4 Clamping & Safety Analysis (Silicon vs. Schottky) . . . 14
- 5.3 Hardware Configuration . . . 15
- 5.4 Firmware Logic . . . 15
- 5.5 Calibration: Mains Hum and Reference Tones . . . 15
  - 5.5.1 Physics of Environmental Coupling . . . 16
  - 5.5.2 Derivation of the Correction Factor . . . 16
  - 5.5.3 Verification Against Crystal-Referenced Tones . . . 16
- 5.6 Final Technical Specifications . . . 17

6 Analysis & Validation **18**
6.1 Experimental Methodology . . . 18
6.2 DAQ Sensitivity & Noise Floor . . . 18
6.3 Gain Structure & Transfer Characteristic . . . 19
6.3.1 The Saturation Gradient . . . 19
6.4 Topology & Harmonic Fingerprint . . . 19
6.4.1 Time Domain: The Soft Knee . . . 20
6.4.2 Frequency Domain: Even-Order Dominance . . . 20
6.5 Active Signal Injection: Transfer Function Analysis . . . 20
6.5.1 The Stream-Mode Discontinuity Problem . . . 21
6.5.2 Chunked Harmonic Fitting . . . 21
6.5.3 Frequency Response . . . 22
6.5.4 Harmonic Distortion vs. Frequency . . . 22
6.5.5 Large-Signal Transfer Curve . . . 22
6.5.6 Cross-Validation Against Burst Capture . . . 24
6.5.7 Even vs. Odd Harmonic Balance . . . 25
6.5.8 Limitations . . . 25

7 Conclusion **26**
7.1 Synthesis: The Recursive Engineering Loop . . . 26
7.2 Summary of Empirical Findings . . . 26
7.3 Phase II: Active Signal Injection . . . 27
7.4 Future Roadmap: The Silicon Revision (v1.0) . . . 27
7.4.1 Hardware Architecture . . . 27
7.4.2 Signal Chain Evolution . . . 28
7.4.3 Programmable Analog Front End (AFE) . . . 28
7.5 Closing Statement . . . 28

A Source Code **29**
A.1 Firmware: Acquisition Loop . . . 29
A.2 Host: Calibration Algorithm . . . 30
A.3 Host: Signal Processing (DSP) . . . 34
A.4 Host: Function Generator Logic (Phase II) . . . 40
A.5 Host: Chunked Sweep Analysis (Phase II) . . . 41

References **48**

Chapter 1

Introduction

1.1 The Systems Engineering Imperative

In modern computational astrophysics, complex systems are modeled with high precision, yet they remain fundamentally virtual. As an undergraduate specializing in data pipelines and Python, I operate within deterministic environments where logic is absolute and variables are strictly controlled. However, the transition from software architecture to physical implementation reveals a critical discontinuity—the "Full-Stack Gap."

In the physical domain, engineering is not merely about logic; it is about managing entropy. Physical systems are subject to stochastic variables—component tolerances, thermal drift, supply chain volatility, and electromagnetic interference—that do not exist in code. This technical report documents a recursive engineering initiative designed to bridge this gap. The primary objective was to fabricate a specific analog device (The Red Llama Overdrive), but the broader pedagogical goal was to apply the rigor of software systems engineering—modularity, regression testing, and version control—to the chaotic reality of hardware fabrication.

1.2 Problem Formulation

The standard hobbyist workflow for electronics fabrication is often characterized by ad-hoc solutions and subjective validation. When analyzed through a systems engineering lens, this approach reveals three critical failure modes that compromise reproducibility and rigorous analysis:

1. **Logistical Entropy:** Unlike software dependencies, which can be resolved instantly via package managers, hardware dependencies are physical and finite. Manual Bill of Materials (BOM) management is nondeterministic and error-prone; a single missing resistor constitutes a critical blocking dependency that can halt a project for weeks.
2. **Infrastructure Debt:** Precision audio circuits utilize high-gain amplification stages that are highly susceptible to power supply noise. Standard consumer-grade Switch Mode Power Supplies (SMPS) inherently introduce high-frequency switching ripple [1]. Relying on generic power infrastructure introduces an uncontrolled variable that aliases into the signal path, invalidating high-fidelity measurements.
3. **Subjective Validation:** In audio electronics, verification is often purely qualitative ("it sounds good"). However, from a physics perspective, "tone" is quantifiable as harmonic distortion and frequency response. Without calibrated instrumentation to visualize the transfer function, the engineering process remains blind, relying on trial-and-error rather than empirical data.

1.3 Methodology: The Recursive Solution

To resolve these deficiencies, I elected to engineer the tooling required to build the device, rather than purchasing off-the-shelf solutions. This ”recursive” approach ensures that every variable in the signal chain—from the procurement of the silicon to the analysis of the output—is understood and controlled.

This document details the design, fabrication, and integration of four interdependent systems:

- **Star Ground (Logistics):** A Python-based logistics engine that treats hardware BOMs as strict data objects. By parsing PDF documentation into structured JSON data, the system eliminates human error from the procurement process, effectively acting as a ”CI/CD pipeline” for physical inventory.
- **Linear Power Regulator (Infrastructure):** A custom, thermal-managed 12V-to-9V linear supply. By utilizing the L7800 series architecture, this subsystem establishes a clean, ripple-free noise floor, ensuring that artifacts observed in the output are generated by the device, not the wall mains.
- **Red Llama Overdrive (Device Under Test):** The primary artifact. This analog circuit utilizes CD4049 CMOS Hex Inverters, originally designed for digital logic, biased into their linear amplification region. This topology is hypothesized to generate soft-clipping saturation characteristics similar to vacuum tube triodes.
- **RP2040 DAQ (Instrumentation):** A custom spectrophotometer and high-speed Analog Front End (AFE). Calibrated against the 60Hz mains frequency, this system enables the empirical verification of the DUT’s harmonic content, moving validation from the subjective to the quantitative.

1.4 Report Structure

The remainder of this document is organized logically following the signal path of the engineering process:

- **Chapter 2** details the software architecture used to manage physical chaos.
- **Chapter 3** outlines the electrical infrastructure required to power the experiment.
- **Chapter 4** describes the fabrication and topology of the analog circuit itself.
- **Chapter 5** explains the design of the digital instrumentation used for analysis.
- **Chapter 6** presents the empirical data, synthesizing the four systems to validate the initial engineering hypotheses.

Every measured value in Chapters 5 and 6 is computed in the Jupyter notebooks in `oscilloscope-rp2040/notebooks/` of the accompanying repository (https://github.com/JacksonFergusonDev/systems-audio-lab). Citations of the form “Notebook `02`, §5” refer to a numbered section of those notebooks: Notebook `01` performs acquisition and calibration, Notebook `02` analyses the guitar captures and validates the instrument, Notebook `03` acquires the test-signal captures, and Notebook `04` analyses them.

Chapter 2

The Logistics Engine (Software)

2.1 Problem Definition: Logistical Entropy

Before a circuit can be soldered, components must be procured. In the domain of DIY electronics, this is a deceptively complex problem. Bill of Materials (BOM) data is distributed across incompatible formats (PDFs, raw text, forum posts), and inventory management is typically manual.

When managing parts for four concurrent builds, the probability of error approaches unity. A single missing 10-cent resistor creates a shipping bottleneck that halts a project for weeks. I determined that manual transcription of BOMs was an unacceptable source of nondeterministic error.[2]

2.2 System Architecture: Star Ground

Star Ground is a Python-based full-stack logistics engine designed to function as the Single Source of Truth for component inventory. In circuit design, a "Star Ground" is the point where all signal paths converge to eliminate noise.[3] In this context, the software eliminates the "noise" of disorganized supply chains.

2.2.1 Design Philosophy: Determinism over AI

A seemingly obvious solution would be to pass PDF documentation to a Large Language Model (LLM) for parsing. However, for procurement, hallucination is a critical failure mode.[4] If an LLM misidentifies a 100kΩ resistor as 10kΩ, the hardware will fail.

Therefore, I rejected probabilistic models in favor of a **Deterministic Regex Engine**. The system uses a Hybrid Strategy for PDF ingestion:

1. **Visual Layout Analysis:** Using `pdfplumber` to extract table structures based on grid lines.[5]
2. **Regex Fallback:** A pattern-matching layer that parses raw text when table extraction fails.
3. **Snapshot Testing:** We use snapshot-based regression tests that compare the parser's current output against stored "golden master" JSON snapshots to detect unintended changes and preserve legacy support. [6, 7]

2.3 Algorithmic Logic: Nerd Economics

The core value of the engine lies in its heuristic inventory buffering. In hardware prototyping, the cost of downtime exceeds the cost of inventory holding. I implemented a "Yield Management Algorithm" that adjusts purchase quantities based on component risk profiles.

The engine calculates the Net Need:

$$\text{Net Need} = \max(0, \text{BOM Qty} - \text{Stock Qty}) \tag{2.1}$$

Safety buffers are applied strictly to the deficit:

- **High Risk / Low Cost (Resistors):** +10 unit buffer. They are fragile, cheap, and easily lost.
- **Critical Silicon (ICs):** +1 unit buffer. Ensures a backup in case of thermal damage during soldering.
- **High Cost (Potentiometers/Switches):** +0 buffer.

```python
def calculate_buffer(component_type: str, cost: float) -> int:
    if component_type == "RESISTOR":
        return 10  # Cheap insurance
    elif component_type == "IC" or component_type == "TRANSISTOR":
        return 1   # Protection against ESD/Heat damage
    elif cost > 2.00:
        return 0   # JIT economics
    return 0
```

Listing 2.1: Heuristic Buffering Logic

2.4 The "Silicon Sommelier"

Beyond simple counting, the engine acts as a domain expert system. It utilizes a lookup table to suggest component upgrades based on audio engineering best practices:

- **Op-Amps:** Detects generic `TL072` chips and suggests Hi-Fi alternatives like `OPA2134` for lower noise floors.[8, 9]
- **Fuzz Logic:** Automatically detects Germanium transistor topologies and flags "Positive Ground" warnings.[10, 11]
- **Texture Generation:** Maps clipping diodes to their sonic equivalents based on Forward Voltage (V_f).[12]

2.5 Generated Artifacts

The final output of the pipeline is not just a shopping list, but a structured build guide. The engine generates "Field Manuals"—PDFs that reorganize the assembly order by component Z-height (Resistors → Sockets → Capacitors). This enforces mechanical stability during soldering, ensuring that low-profile components are not obstructed by taller ones.

2.6 Validation

This software was used to procure and manage the inventory for the Red Llama build (Chapter 4) and the Power Regulator (Chapter 3). In practice, both builds were completed without a missing-part delay or a surplus of high-value components.

Chapter 3

Infrastructure: Linear Power Supply

3.1 Project Motivation

With the logistics pipeline (Chapter 2) operational, attention turned to the physical infrastructure required to support the Device Under Test (DUT). Audio circuits are notoriously sensitive to power supply noise. Standard consumer-grade "wall warts" are typically Switch Mode Power Supplies (SMPS), which operate by rapidly switching a transistor on and off to maintain voltage. This switching frequency often leaks into the audio path, manifesting as high-frequency whine or aliasing artifacts.[1]

Furthermore, I possessed a surplus of 12V power bricks, but the standard for guitar pedals is 9V Center-Negative. Rather than purchasing a dedicated supply, I elected to engineer a **Linear Voltage Regulator**. This served a dual purpose: providing a clean, ripple-free power source for the DUT and serving as a low-risk "Hello World" for discrete fabrication before attempting the audio circuit.

3.2 Theory of Operation

The circuit relies on the **L7809CV**, a classic series linear regulator.[13] Unlike switching regulators, a linear regulator operates as a variable resistor, effectively "burning off" excess voltage as waste heat to maintain a constant output.[13] While less efficient than SMPS, the output is practically free of high-frequency ripple, making it ideal for audio applications.

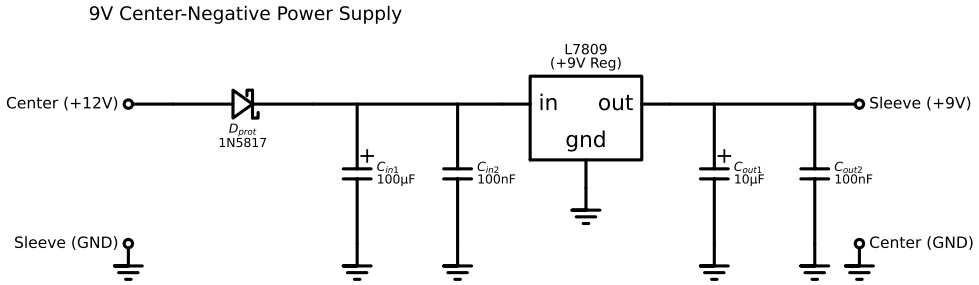


Figure 3.1: **Linear Regulator Schematic.** The circuit utilizes a standard L7809 topology with added input/output bulk capacitance and a reverse-bias protection diode (D_{prot}) to prevent damage from negative voltage transients.

3.3 Thermal Management Strategy

The primary engineering constraint in linear regulation is heat dissipation. The power dissipated (P_d) is a function of the voltage drop and load current:

$$P_d = (V_{in} - V_{out}) \times I_{load} \tag{3.1}$$

Given an input of 12V and a target output of 9V, the regulator drops 3V across the junction.

- **Scenario A (Red Llama):** The analog DUT's current draw was not measured; CMOS pedals of this kind typically draw well under 10mA, which would give $P_d < 0.03$W.
- **Scenario B (Future Proofing):** Digital pedals (reverbs/delays) draw far more; a 400mA load is taken as the design case.

For Scenario B, the dissipation becomes significant:

$$P_{max} \approx 3\text{V} \times 0.4\text{A} = 1.2\text{W} \tag{3.2}$$

A standard TO-220 package has a thermal resistance (θ_{JA}) of $\approx 65^\circ$C/W.[13] Without a heatsink, a 1.2W load would result in a temperature rise of:

$$\Delta T = 1.2\text{W} \times 65^\circ\text{C/W} = 78^\circ\text{C} \tag{3.3}$$

Assuming an ambient temperature of 25°C, the silicon junction would reach 103°C. While within the absolute maximum (125°C), this is dangerously hot for an enclosed ABS plastic chassis.[13]

3.3.1 The Solution

To mitigate this, I implemented a two-stage thermal solution:

1. **Conduction:** A bolt-on aluminum heatsink was attached to the TO-220 package to lower the thermal resistance.
2. **Convection:** Ventilation shafts were drilled into the top of the enclosure to establish a convection current, preventing the ABS box from acting as a thermal insulator.

3.4 Component Selection

The Bill of Materials, generated via Star Ground, included specific selections for noise filtering and protection:

- **Input Filtering:** A 100μF electrolytic capacitor handles bulk low-frequency ripple from the 12V source, while a parallel 100nF polyester capacitor filters high-frequency transients.[13]
- **Reverse Polarity Protection:** A **1N5817 Schottky Diode** was selected over a standard 1N4001 silicon diode.[14, 15] The Schottky's lower forward voltage drop ($V_f \approx 0.45$V at its rated 1A, and ≈ 0.3V at the few milliamps a pedal draws) preserves more of the input voltage headroom compared to silicon ($V_f \approx 0.7$V).[14, 15]

Des.	Value	Component	SKU	Price	Qty	Total	Notes
U1	L7809	L7809CV-DG Reg 9V 1.5A	A-1597	$0.25	1	$0.25	TO-220
D_{prot}	1N5817	Schottky Diode 1A 20V	A-159	$0.06	1	$0.06	Low V_f
C_{in1}	100μF	35V Radial Electrolytic	A-987	$0.03	1	$0.03	Bulk In
C_{out1}	10μF	50V Radial Electrolytic	A-4554	$0.02	1	$0.02	Bulk Out
$C_{f1,f2}$	100nF	100V 5% Poly Box	A-564	$0.10	2	$0.20	HF Filter
J1	2.1mm	DC Power Jack (Fem)	A-2237	$0.13	1	$0.13	Panel Mnt
P1	2.1mm	DC Plug (Male) Pigtail	A-6806	$0.15	1	$0.15	Output
–	Box	Black Plastic Box 03	A-2383	$2.50	1	$2.50	ABS
–	Heatsink	TO-220 10-Fin 1”	A-1512	$0.24	1	$0.24	For U1
–	M3	Hex Socket Cap Screw	A-6379	$0.10	1	$0.10	Mounting
			PROJECT TOTAL			**$3.68**	

Table 3.1: **Bill of Materials.** All components sourced from Tayda Electronics to minimize shipping overhead. Prices reflect unit costs at time of purchase.

Chapter 4

Device Under Test: Red Llama Overdrive

4.1 Circuit Topology

With the logistics managed (Chapter 2) and the power cleaner (Chapter 3), fabrication began on the primary objective: The **Red Llama Overdrive**.

This circuit is topologically distinct among guitar pedals. While most overdrives utilize Operational Amplifiers (Op-Amps) with diode clipping in the feedback loop, the Red Llama utilizes the **CD4049 CMOS Hex Inverter**.[16]

Originally designed for digital logic (translating 1s and 0s), the CD4049 consists of internal push-pull MOSFET pairs.[16] By applying negative feedback, the digital gates are forced into a linear amplification region. When driven hard by a guitar signal, the MOSFETs hit the power supply rails, creating a soft, rounded saturation curve that mimics the behavior of vacuum tubes, eventually squaring off into a hard fuzz at maximum gain.

4.2 Procurement & Fabrication

The build utilized the output artifacts from the **Star Ground** engine. The "Field Manual" PDF dictated the assembly order based on Z-height, ensuring that low-profile resistors were soldered before the taller IC sockets and capacitors. This procedural rigor minimized the mechanical frustration often associated with high-density Perfboard layouts.

4.3 Engineering Deviation: The Diode Mod

While the build largely followed the standard Way Huge Electronics schematic, I introduced a modification to the power section to maximize dynamic range.

The reference design calls for a **1N4001** silicon diode at position D2 for reverse-polarity protection.[17] As noted in Chapter 3, silicon diodes exhibit a standard forward voltage drop of ≈ 0.7V.[15] In a 9V circuit, this drop is non-trivial:

$$V_{rail} = V_{source} - V_{diode} = 9.0\text{V} - 0.7\text{V} = 8.3\text{V} \tag{4.1}$$

The CD4049's headroom is strictly limited by the supply rails. Losing 0.7V reduces the clean headroom before clipping occurs. To recover this, I substituted the 1N4001 with a **1N5817 Schottky Diode**, whose forward drop is ≈ 0.3V at the few milliamps the pedal draws.[14]

$$V_{rail(mod)} = 9.0\text{V} - 0.3\text{V} = 8.7\text{V} \tag{4.2}$$

By recovering $\approx$ 0.4V of supply voltage, the CMOS inverters have slightly more headroom. This translates to increased dynamic range and a cleaner transient response before the onset of saturation.

4.4 The Validation Gap

Upon completion, the device passed the "Smoke Test" and produced sound. Subjectively, the tone was rich and harmonically complex. However, as a scientist, subjective listening is insufficient. I required empirical verification of the saturation characteristics.

I did not own an oscilloscope. Rather than buying one, I realized that the surplus **RP2040** microcontroller I had procured (via the Star Ground logistics run) possessed the theoretical capability to act as a high-speed digitizer.[18] This led to the development of the custom DAQ system detailed in the following chapter.

Chapter 5

Instrumentation: RP2040 DAQ

5.1 System Architecture

To validate the saturation behavior of the Red Llama (Chapter 4), I required a Data Acquisition (DAQ) system capable of capturing audio frequencies with high fidelity. The **RP2040** microcontroller (dual-core ARM Cortex-M0+) was selected as the engine due to its flexible I/O and low cost.[19]

The system operates on a **"Store-and-Forward"** architecture. Unlike typical streaming applications that push data continuously (and suffer from jitter due to USB packetization), this system prioritizes sample rate stability. It captures a precise burst of data into a pre-allocated ring buffer in RAM, then transmits the binary blob over USB Serial only *after* acquisition is complete.[19] This decouples the strict timing requirements of the sampling loop from the non-deterministic latency of the host workstation.

5.2 Analog Front End (AFE) Theory

The RP2040's internal ADC is limited to a 0 V to 3.3 V unipolar range.[19] Direct connection to the Red Llama (which outputs high-amplitude bipolar AC signals) would result in negative voltage clipping and potential silicon latch-up. A "Universal" Signal Conditioning Interface was designed to bridge this gap.

5.2.1 Input Protection I

A series resistor $R_{prot} = 10\,\text{k}\Omega$ is placed immediately at the input jack. This limits the current flowing into the subsequent clamping stage during high-voltage transients or accidental connection to modular synth rails (± 12 V), preventing thermal destruction of the protection diodes.

5.2.2 High-Pass Filter (AC Coupling)

For audio signals, the DC component is blocked using a polyester film capacitor $C_{ac} = 220\,\text{nF}$. The corner frequency (f_c) is determined by the input impedance of the subsequent stage. In **High-Z / Guitar Mode**, the input impedance is dominated by the bias injection resistor $R_{inject} = 220\,\text{k}\Omega$.

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \cdot 220\,\text{k}\Omega \cdot 220\,\text{nF}} \approx 3.29\,\text{Hz} \tag{5.1}$$

This provides a flat frequency response well below the audible floor (20 Hz).

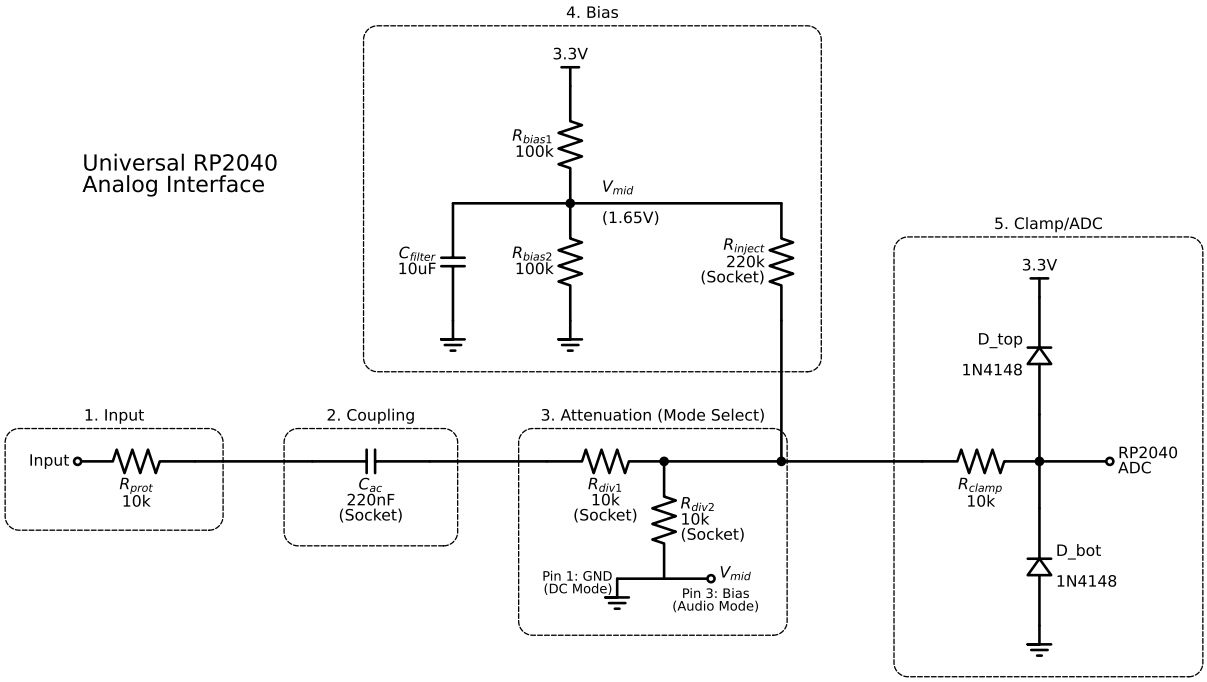


Figure 5.1: **System Schematic.** The signal path flows left-to-right: Protection Stage → AC Coupling → Attenuation/Bias Network → ADC Input.

5.2.3 Bias & Switched Reference Topology

The RP2040 ADC requires a signal centered at 1.65 V (half-rail) to read AC waveforms.[19] A stiff voltage divider (100 kΩ pair) filtered by a 10 μF electrolytic capacitor generates a stable quiet rail (V_{mid}).

A **Switched Reference Topology** is used to solve impedance conflicts via a physical jumper (JP1):

- **Audio Mode:** The jumper connects the bottom of the attenuation network to V_{mid} (1.65 V). The signal "sees" virtual ground, preserving the gain ratio while biasing the signal center point.
- **DC Mode:** The jumper connects to GND, forming a standard resistive divider for unipolar sensing ($0 - 5$ V sensors).

5.2.4 Clamping & Safety Analysis (Silicon vs. Schottky)

A critical design decision was the selection of **1N4148 Silicon Diodes** for rail clamping over standard Schottky diodes.

Standard low-voltage protection often utilizes Schottky diodes (e.g., BAT54) due to their low forward voltage drop ($V_f \approx 0.3$ V), which keeps signal excursions strictly within the datasheet absolute maximums ($V_{DD}+0.5$ V).[20] However, Schottky diodes exhibit significant reverse leakage current ($I_R \approx 1$–5 μA).[20] Given the High-Z nature of this AFE (specifically the 220 kΩ bias injection), a leakage current of just 1 μA would introduce a DC offset error of $V_{err} = 220$ mV, significantly reducing dynamic range.

By selecting **1N4148 Silicon Diodes**, we prioritize precision ($I_R < 25$ nA).[21] The trade-off is a higher forward voltage ($V_f \approx 0.7$ V).[21] During an over-voltage event, the clamping node reaches:

$$V_{clamp} = V_{DD} + V_f = 3.3\,\text{V} + 0.7\,\text{V} = 4.0\,\text{V} \tag{5.2}$$

This exceeds the RP2040 absolute maximum rating of 3.8 V.[19] However, the safety of the device is guaranteed by the current-limiting resistor R_{clamp} (10 kΩ). If the external signal forces the node to 4.0 V,

the internal ESD diodes of the ADC will begin to conduct at approx 3.8 V. The resulting current injected into the pin is:

$$I_{inj} = \frac{V_{clamp} - V_{internal}}{R_{clamp}} = \frac{4.0\,\text{V} - 3.8\,\text{V}}{10\,\text{k}\Omega} = 20\,\mu\text{A} \tag{5.3}$$

This injection current of 20 μA is extremely small, and the input is protected primarily by current limiting.

5.3 Hardware Configuration

The board functionality is determined by the presence of socketed components and the position of the 3-pin jumper. This allows the same physical board to serve as a Guitar Interface, Line Level Input, or Sensor Reader.

Mode	Application	Jumper (JP1)	R_{div1}/R_{div2}	R_{inject}
Audio (Std)	Synths, Pedals	Pin 2-3 (V_{mid})	10k / 10k	*Empty*
Guitar (High-Z)	Passive Inst.	Pin 2-3 (V_{mid})	Wire / *Empty*	220k
DC / Sensor	0-5V Signals	Pin 1-2 (GND)	10k / 10k	*Empty*

Table 5.1: Jumper Settings and Component Population Guide

5.4 Firmware Logic

The firmware is written in MicroPython to leverage rapid development, but standard Python is too slow for high-speed sampling. To maximize throughput, the critical capture loop uses the `@micropython.native` decorator.[22]

```
@micropython.native
def capture_burst(adc_obj, buf, size: int):
    # Compiles to ARM Thumb machine code
    for i in range(size):
        buf[i] = adc_obj.read_u16()
```

Listing 5.1: Native Emitter Capture Loop

This instructs the compiler to emit ARM Thumb machine code directly.[22] While the method lookup still incurs overhead, the loop mechanics (increment, compare, jump) become native CPU instructions. This optimization allowed the system to achieve a stable sample rate of $\approx$ 100 kSps, far exceeding the initial 40 kSps target.

5.5 Calibration: Mains Hum and Reference Tones

Accurate frequency domain analysis requires a precise knowledge of the sampling rate (F_s). While the RP2040 hardware clock is stable, the firmware architecture relies on a MicroPython polling loop running on the CPU rather than a DMA-driven hardware timer. Consequently, the effective sample rate is determined by the interpreter's execution overhead, which creates a deterministic but unknown latency.

To resolve this without external signal generators, the system was calibrated using a ubiquitous, high-stability reference standard: the North American Power Grid (60.00 Hz).

5.5.1 Physics of Environmental Coupling

The calibration signal was acquired by the user touching the input probe tip. This technique exploits the physics of **capacitive coupling**. The physical system can be modeled as a capacitive voltage divider:

1. **Source (V_{grid}):** The 120 V_{rms} wiring in the walls creates an oscillating electric field ($\omega = 2\pi \cdot 60$).
2. **Coupling Impedance (Z_c):** The user's body acts as a conductive plate separated from the wiring by a dielectric (air/insulation). This forms a stray capacitance ($C_{stray} \approx 10 - 100$ pF).
3. **Load Impedance (Z_{in}):** The AFE input resistance (approx. 220 kΩ in High-Z mode).

The voltage appearing at the ADC (V_{adc}) is determined by the ratio of the input impedance to the total path impedance. Given that the reactance of a 50 pF gap at 60 Hz is extremely high (≈ 53 MΩ), the circuit forms a high-impedance attenuator. However, because the source potential is high (≈ 170 V_{pk}), even this tiny coupling ratio yields a measurable signal in the millivolt range.

5.5.2 Derivation of the Correction Factor

Because the frequency axis of an FFT scales linearly with F_s, a reference of known frequency f_{ref} that measures f_{meas} under an assumed rate $F_{assumed}$ gives the true rate directly:

$$F_{real} = F_{assumed} \cdot \frac{f_{ref}}{f_{meas}} \tag{5.4}$$

The first-pass calibration applied this to the mains hum, locating the 60 Hz peak with a weighted average of the FFT bins around it, and gave $F_s = 97{,}859.6$ Hz (Notebook 01, first cell).

5.5.3 Verification Against Crystal-Referenced Tones

Mains hum is a weak reference for a single burst. The grid wanders by a few hundredths of a hertz, a 16,384-sample burst holds only ≈ 10 cycles, and the coupled hum is just 2.4 mV peak, 2.3 dB above the noise floor. The Phase II test tones (Section 6.5) are a far better reference: a sound card derives them from a crystal clock, typically accurate to ±50 ppm. Each was fitted by least squares, which for a steady tone resolves frequency far below the ≈ 6 Hz FFT bin (Notebook 02, §5b):

Reference	Assumed F_s (Hz)	Implied F_s (Hz)
1 kHz sine, laptop direct	97,793.1	97,683.3 ± 0.0
1 kHz sine, via Red Llama	97,793.1	97,683.2 ± 0.3
100 Hz triangle, laptop direct	97,793.1	97,688.9 ± 0.6
60 Hz mains hum	97,859.6	97,570.3 ± 45.6

Table 5.2: Sample rate implied by each reference signal. Source: Notebook 02, §5b.

The three crystal-referenced tones agree within 58 ppm, giving $F_s = 97.69$ kHz ± 0.01%. The mains calibration was 0.18% high, and the hardware default used during the Phase II captures (97,793.1 Hz) 0.11% high; the mains capture is consistent with the tones within its large uncertainty. All analysis

in Chapter 6 uses the measured rate. A rate error scales every frequency axis by the same factor, so harmonic ratios, THD and decibel levels are unaffected by it.

5.6 Final Technical Specifications

The following specifications reflect the measured performance of the commissioned device.

Parameter	Value / Note
ADC Resolution	12-bit (Scaled to 16-bit integer)
Sample Rate (F_s)	97.69 kHz ± 0.01% (Crystal-referenced)
Bandwidth	DC – 48.8 kHz (Nyquist Limit)
Buffer Depth	16,384 Samples (≈ 168 ms duration)
Stream Mode	1,024-sample blocks; ≈ 75% duty cycle (untimed gaps)
Spectral Resolution	5.96 Hz per FFT bin
Input Range	0–3.3 V (Unipolar) / ±3.3 V (AC Coupled)
Noise Floor	1.29 mV RMS (input open)
Dynamic Range	59.1 dB full-scale sine to noise (9.5 effective bits)

Table 5.3: Measured System Performance. Source: Notebook 02, §5b (sample rate), Notebook 02, §5 (noise floor, dynamic range), Notebook 04, §3 (stream-mode duty cycle); bandwidth, buffer duration and bin width follow from F_s and N.

Chapter 6

Analysis & Validation

6.1 Experimental Methodology

To characterize the Red Llama Overdrive, the device was inserted into the signal chain between a reference source (Electric Guitar, Neck Pickup) and the RP2040 DAQ. The data acquisition pipeline utilized the system's "Science Mode," executing a burst capture of $N = 16,384$ samples at a calibrated rate of $F_s \approx 97.7$ kSps (Section 5.5).

The analysis pipeline was implemented in Python using `numpy` and `scipy.signal`.[23, 24] Crucially, to mitigate spectral leakage caused by the finite sampling interval, a **Hann Window** function ($w(n) = 0.5 - 0.5\cos(\frac{2\pi n}{N-1})$) was applied to the raw time-domain buffer prior to the Fast Fourier Transform (FFT).[25, 26] This preprocessing step ensures that the side-lobes of high-amplitude fundamental frequencies do not obscure the lower-amplitude harmonic distortion products.[27]

The swept measurements of Section 6.5 additionally use the firmware's stream mode, which requires a different analysis method described there.

6.2 DAQ Sensitivity & Noise Floor

Before characterizing the Device Under Test (DUT), the instrumentation itself was validated to quantify the available dynamic range. Figure 6.1 illustrates the spectral density of the system with the input grounded.

With the input open, the system exhibits a read noise of 1.293 mV RMS, about 1.6 ADC steps. An independent estimate comes from the mains-hum calibration capture: after fitting and removing the hum, the residual "thickness" of the waveform is 1.306 mV RMS, agreeing within 1%.

The usable range of the instrument is its **dynamic range**: the largest clean signal over the noise floor. The ADC window (0–3.3 V) admits a full-scale sine of 1.65 V peak, i.e. 1.167 V_{rms}:

$$\mathrm{DR}_{dB} = 20 \cdot \log_{10}\left(\frac{V_{FS,rms}}{V_{noise,rms}}\right) = 20 \cdot \log_{10}\left(\frac{1.167}{1.293 \times 10^{-3}}\right) \approx 59.1\,\mathrm{dB} \tag{6.1}$$

This corresponds to 9.5 effective bits of the ADC's 12 (an ideal 12-bit converter reaches 74.0 dB). Mains pickup is negligible: the open-input spectrum contains a 60 Hz line of only ≈ 0.05 mV amplitude. All values are computed in Notebook 02, §5.

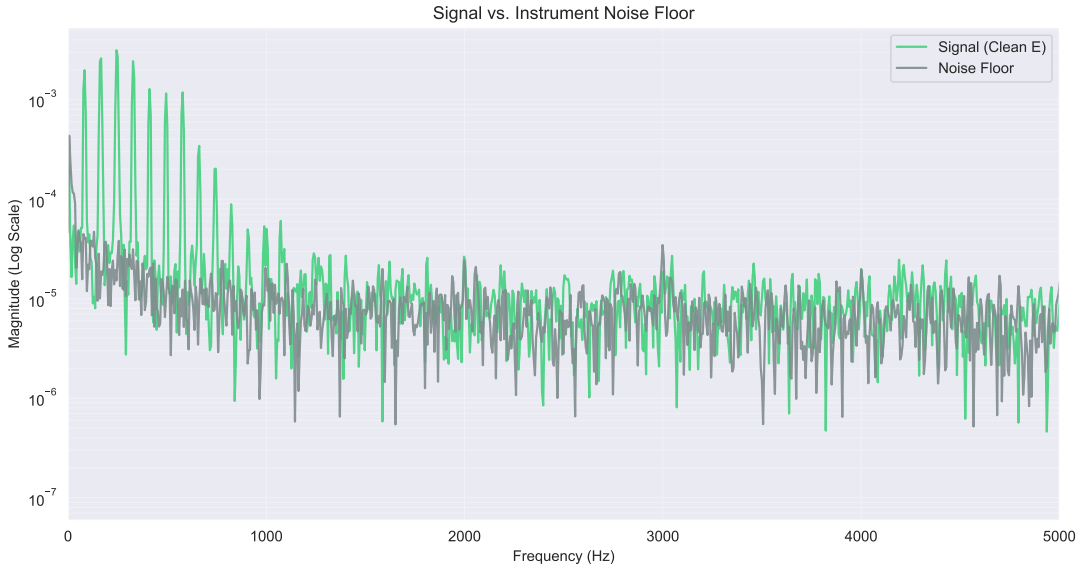


Figure 6.1: **DAQ Sensitivity Analysis.** The system exhibits a read noise of 1.29mV RMS. Note the clean separation between the signal (green) and the noise floor (gray) below 1kHz. Full-scale dynamic range is 59.1dB. Source: Notebook 02, §5.

6.3 Gain Structure & Transfer Characteristic

The Red Llama operates as a high-gain pre-amplifier. To visualize the amplification factor, the input signal (49mV$_{pp}$) was plotted against the output on a unified voltage scale (Figure 6.2). The clean note was captured at 1.0× DAQ gain and the Red Llama output through the 0.5× divider; both are referred to the instrument / pedal output.

The output reaches 3.54V$_{pp}$, a peak-to-peak gain of $\approx$ 72× (37.1dB; Notebook 02, §3). The figure is approximate: the clean and Red Llama captures are separate plucks, and the divider ratio is nominal.

6.3.1 The Saturation Gradient

The visual density of the output waveform indicates the physical mechanism of the distortion. Plotting V_{out} as a function of V_{in} reveals the device's **Transfer Characteristic**.

- **High-Gain Region:** Around zero, the output changes steeply with the input.
- **Saturation Region:** As the output approaches its limits, the slope dV_{out}/dV_{in} falls toward zero continuously, without a sharp corner.

Mathematically, this distinguishes the CMOS topology from standard silicon diode clipping. While diodes exhibit a sharp discontinuity ($dV/dt \to \infty$) at the forward voltage threshold (V_f), the CMOS inverter behaves as a continuous hyperbolic tangent (tanh) function, resulting in a smoother transition into saturation.

6.4 Topology & Harmonic Fingerprint

The primary engineering hypothesis of this project was that the CD4049 CMOS Hex Inverter, despite being a digital logic chip, behaves analogously to a Vacuum Tube Triode when biased into its linear region. Figure 6.3 presents the phase-locked comparison of the Clean vs. Driven signal.

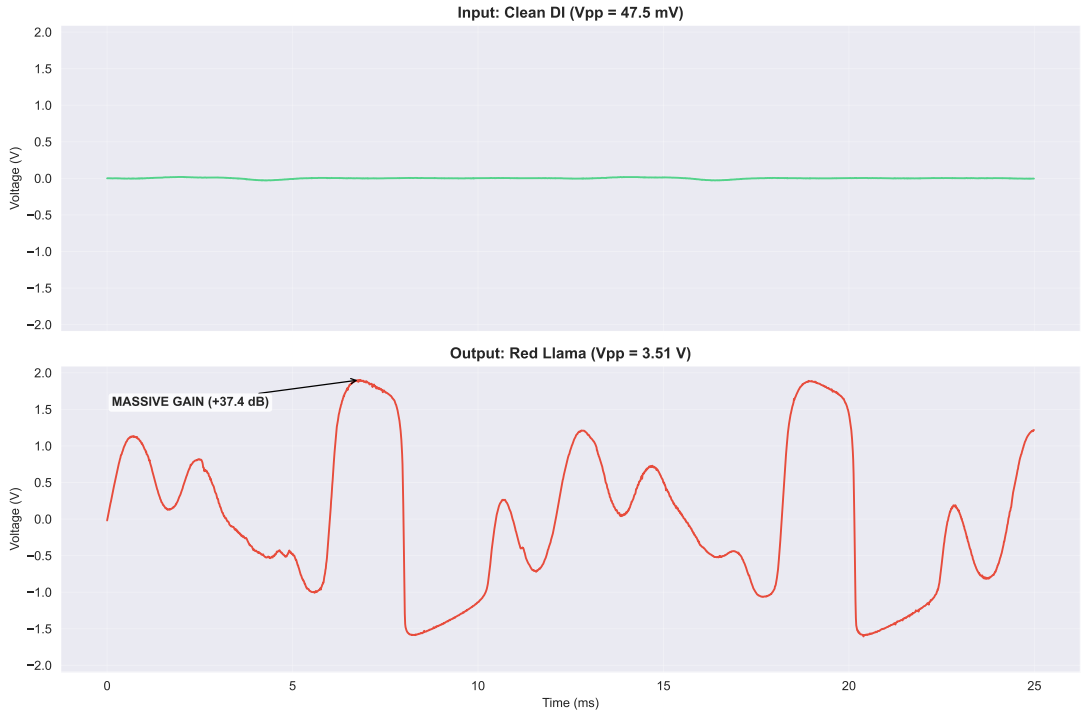


Figure 6.2: **Gain Stage Characterization.** The circuit provides ≈ 37dB of gain. The squaring of the waveform indicates the onset of heavy saturation, where the MOSFETs collide with the power supply rails. Source: Notebook 02, §4; over the plotted 25 ms window the gain is 37.4dB.

6.4.1 Time Domain: The Soft Knee

As seen in the left panel of Figure 6.3, the waveform peaks exhibit a "Soft Knee" compression. This confirms the continuous transfer function described in Section 6.3. The peaks are not sheared off flatly; they are rounded. This preserves the dynamic "feel" of the instrument, allowing the player to control the distortion amount via pick attack velocity.

6.4.2 Frequency Domain: Even-Order Dominance

The spectral density analysis (Figure 6.3, Right) reveals the harmonic signature of this topology (Notebook 02, §7).

- **Even Harmonics (2nd, 4th):** There is a distinct, high-amplitude presence of the 2nd harmonic (the Octave). This asymmetry is mathematically consistent with the single-ended transfer function of vacuum tubes and is responsible for the perceived "warmth" of the circuit.
- **Odd Harmonics (3rd, 5th):** While present, they do not completely dominate the spectrum as they would in a symmetrical diode-clipping circuit.

The clear resolution of these harmonic peaks (5.96 Hz/bin) validates the decision to use a custom Hann-windowed FFT pipeline rather than a standard real-time streaming view.

6.5 Active Signal Injection: Transfer Function Analysis

The guitar captures above characterize the DUT at a single, uncontrolled operating point. To measure its behavior across frequency, the host laptop's audio DAC was used as a signal generator, driving the

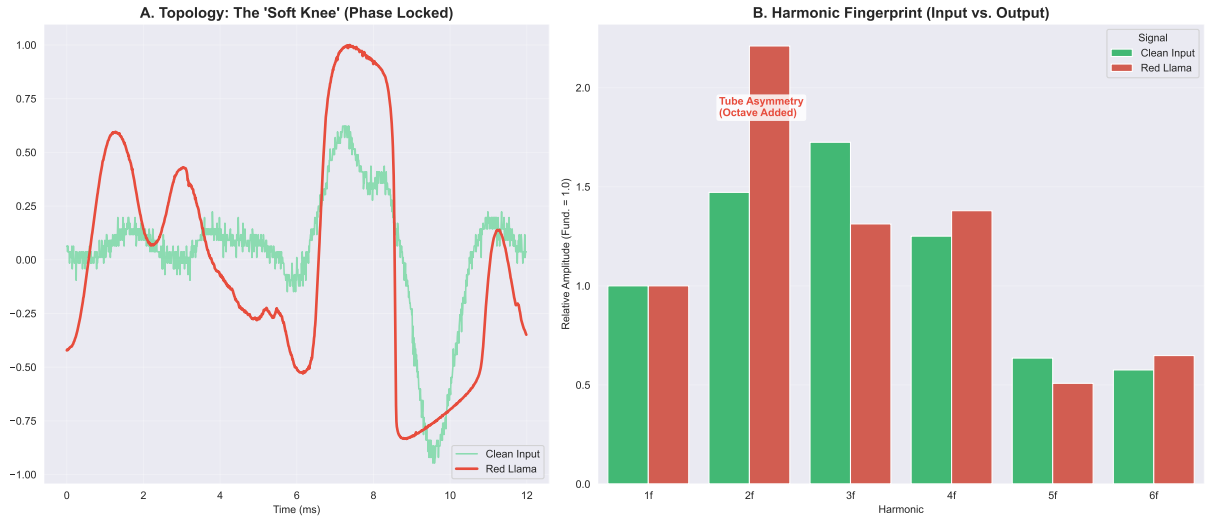


Figure 6.3: **Empirical Validation of CMOS Topology.** (A) Phase-locked time domain analysis reveals 'soft knee' compression characteristics distinct from hard diode clipping. (B) Harmonic fingerprinting confirms a dominant 2nd harmonic (octave) and strong even-order series, consistent with the asymmetrical saturation of vacuum tube triodes. Source: Notebook 02, §9.

DUT through a TRS-to-TS adapter cable. Four captures were taken (Table 6.1).

Capture	Stimulus	Signal Path	DAQ Mode
Source sweep	Log sweep, 20 Hz–20 kHz, 5 s	Laptop → DAQ (loopback)	Stream
DUT sweep	Log sweep, 20 Hz–20 kHz, 5 s	Laptop → Red Llama (max gain) → DAQ	Stream
Triangle	100 Hz triangle	Source and DUT captured separately	Burst
Sine	1 kHz sine	Laptop → Red Llama → DAQ	Burst

Table 6.1: Active signal injection captures. Source: Notebook 03 (acquisition), Notebook 04, §2.

6.5.1 The Stream-Mode Discontinuity Problem

A 5 s sweep does not fit in the 16,384-sample ($\approx$ 168 ms) burst buffer, so the sweeps were captured in the firmware's stream mode. Stream mode is request–response: the host requests a 1,024-sample block, the RP2040 samples it (10.5 ms), transmits it over USB, and waits for the next request. Analysis of the captures shows a block period of $\approx$ 13.9 ms, so roughly 25% of wall-clock time falls into untimed gaps between blocks. Concatenated, the blocks form a signal with a phase discontinuity at every block boundary (sample-to-sample jumps 100–166× larger than elsewhere; Notebook 04, §3).

The standard technique for swept-sine measurement, Exponential Sine Sweep (ESS) deconvolution,[28] convolves the whole response with an inverse sweep and therefore requires a phase-continuous recording. With 357 untimed gaps it is not applicable to these captures.

6.5.2 Chunked Harmonic Fitting

Although the capture as a whole is discontinuous, each block is gap-free, and a 5 s log sweep is quasi-stationary over 10.5 ms (the frequency changes by $< 1.5\%$ within a block; Notebook 04, §3). The analysis therefore treats each block as an independent measurement:

1. **Timing Recovery.** The sweep's instantaneous frequency is $f(t) = f_1 R^t$ with $R = (f_2/f_1)^{1/T}$. The dominant frequency of each block (from a zero-padded, Hann-windowed FFT) implies a sweep time t_k. A robust line fit $t_k = t_0 + k\Delta$ recovers the block period ($\Delta = 13.88$ ms source, 14.03 ms DUT; Notebook 04, §3) and extrapolates to low-frequency blocks where a direct estimate is unreliable.

2. **Harmonic Model.** With τ the time from the block centre and f_k its centre frequency, the exact sweep phase is

$$\phi_k(\tau) = \frac{2\pi f_k}{\ln R}\,(R^\tau - 1)\,, \tag{6.2}$$

   and each block is fitted by linear least squares to

$$x(\tau) = c_0 + \sum_{h=1}^{5} \left[a_h \cos\left(h\,\phi_k(\tau)\right) + b_h \sin\left(h\,\phi_k(\tau)\right)\right], \tag{6.3}$$

   giving the amplitude of harmonic h as $A_h = \sqrt{a_h^2 + b_h^2}$.

3. **Normalization.** The DUT/source ratio of fundamental amplitudes cancels the response of the laptop DAC and the DAQ front end. The source capture's own harmonics define the distortion floor of the measurement chain.

Blocks holding less than one cycle ($f < f_s/1024 \approx 95$ Hz) are excluded. The method was verified against synthetic stream captures with random gaps and timing jitter, recovering a known first-order low-pass response within 0.5 dB and the analytic harmonics of a polynomial nonlinearity within 1 dB (`tests/test_chunked_sweep.py`). The implementation is `sysaudio.chunked_sweep` (Appendix A.5).

6.5.3 Frequency Response

The DUT is flat within 0.5 dB from 97 Hz to 3.4 kHz, then rolls off gently: −1 dB at 5.8 kHz, −2 dB at 11.9 kHz and −2.8 dB at 19.1 kHz (Figure 6.4, top; Notebook 04, §4). The shallow slope is consistent with a single-pole low-pass formed by the inverters' feedback network, with its corner near the top of the audio band.

This measurement was made at maximum gain, where the circuit is heavily saturated. A saturated stage's output level is set largely by its clipping ceiling rather than by its input, which tends to flatten the apparent response. The small-signal response, including the low-frequency corner of the coupling capacitors (below the ≈ 95 Hz analysis floor), remains to be measured at low gain.

6.5.4 Harmonic Distortion vs. Frequency

The harmonic profile is essentially independent of frequency up to ≈ 5 kHz (Figure 6.4, bottom; Notebook 04, §4): median levels over 200 Hz–5 kHz are $H_2 = -12.9$, $H_3 = -12.8$, $H_4 = -15.3$ and $H_5 = -23.7$ dBc. H_2 and H_3 are equal, but H_4 sits 8 dB above H_5, an early sign that the series is not symmetric (Section 6.5.7). Above 5 kHz the harmonics themselves fall into the treble roll-off, and H_2 drops to −18.3 dBc at 19.1 kHz. The source loopback's harmonics sit at ≈ −47 dBc throughout, more than 30 dB below the DUT's, confirming that the measured distortion is produced by the Red Llama and not by the measurement chain.

6.5.5 Large-Signal Transfer Curve

Plotting output against input for a 100 Hz triangle (Figure 6.5; Notebook 04, §5) traces the saturation curve directly. Its shape refines the earlier findings in three ways:

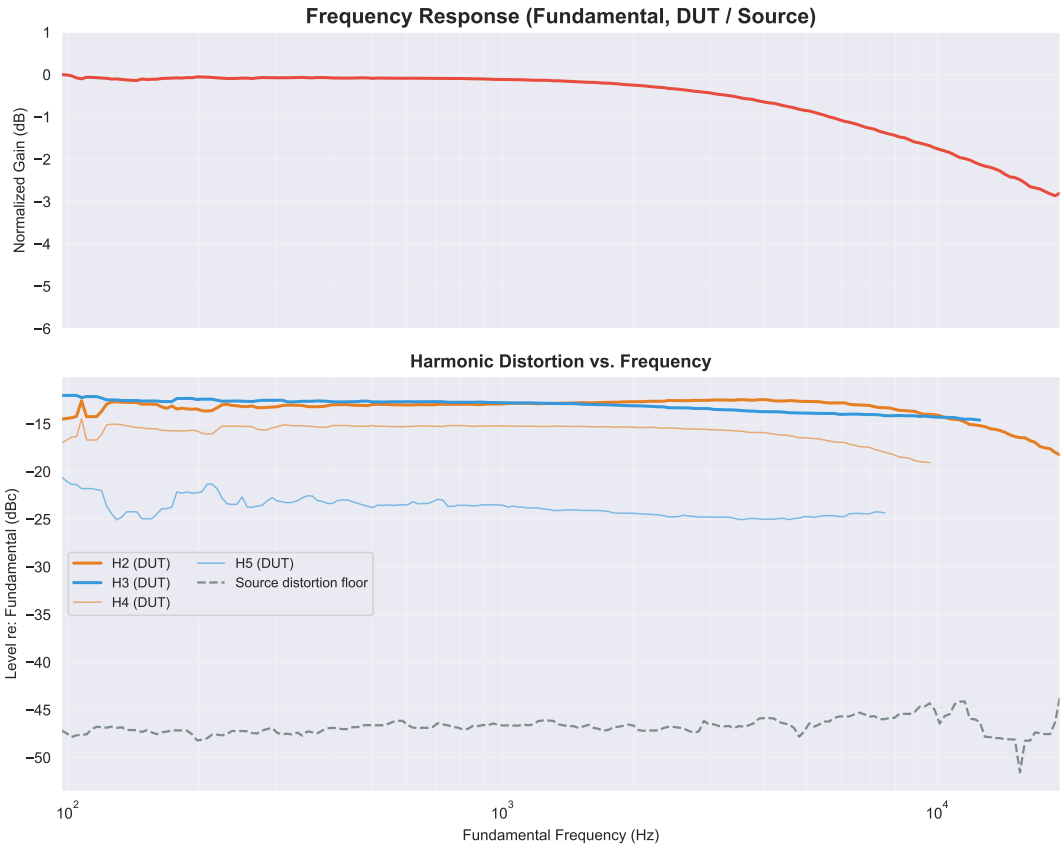


Figure 6.4: **Swept Transfer Function (Max Gain).** Top: fundamental magnitude response, DUT/source, normalized to 0 dB. Bottom: harmonics 2–5 relative to the fundamental, against the source loopback's distortion floor (dashed). Recovered from stream-mode captures by chunked harmonic fitting. Source: Notebook 04, §4.

- **Soft Saturation:** Gain is steep around zero and decreases smoothly toward the rails, with no sharp clipping threshold.
- **Asymmetry:** The pedal output peaks at +2.11 V but only −1.81 V (ratio 1.17; 1.35 for the 1 kHz sine), referred through the DAQ's 0.5× divider. Unequal clipping of the two half-cycles is the physical origin of the even harmonics.
- **Memory:** The rising and falling halves trace different paths, forming a loop rather than a single curve. The coupling capacitors add phase shift at 100 Hz, visible as "droop" on the flat tops. Part of the loop may be residual misalignment, since source and DUT were captured separately.

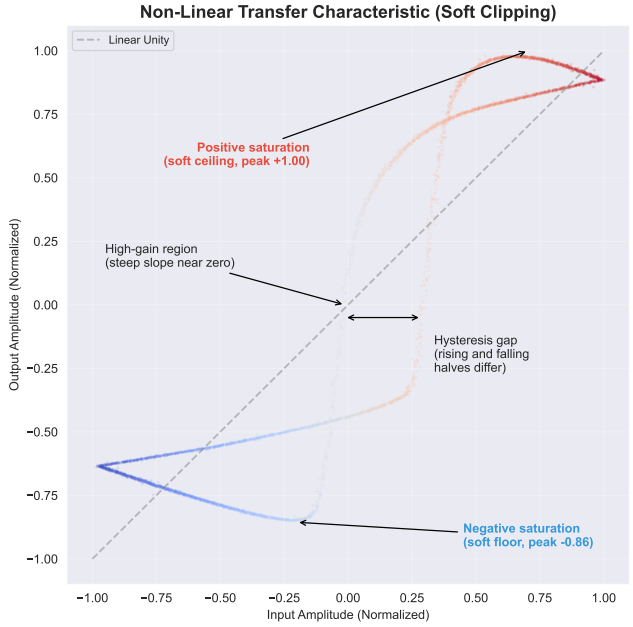


Figure 6.5: **Large-Signal Transfer Characteristic.** DUT output vs. source input for a 100 Hz triangle, both normalized. The sigmoid is asymmetric and opens into a loop due to capacitive coupling. Source: Notebook 04, §5.

6.5.6 Cross-Validation Against Burst Capture

A 1 kHz sine captured in burst mode is gap-free and independent of the sweep analysis. Its harmonic spectrum agrees with the chunked sweep at 1 kHz to within 0.2 dB (Table 6.2, Figure 6.6; Notebook 04, §6), validating the method on real data.

Harmonic	Burst Sine (dBc)	Chunked Sweep (dBc)
H_2	−12.9	−12.9
H_3	−12.8	−12.8
H_4	−15.2	−15.2
H_5	−23.7	−23.5

Table 6.2: Harmonic levels at 1 kHz from two independent capture modes. Source: Notebook 04, §6.

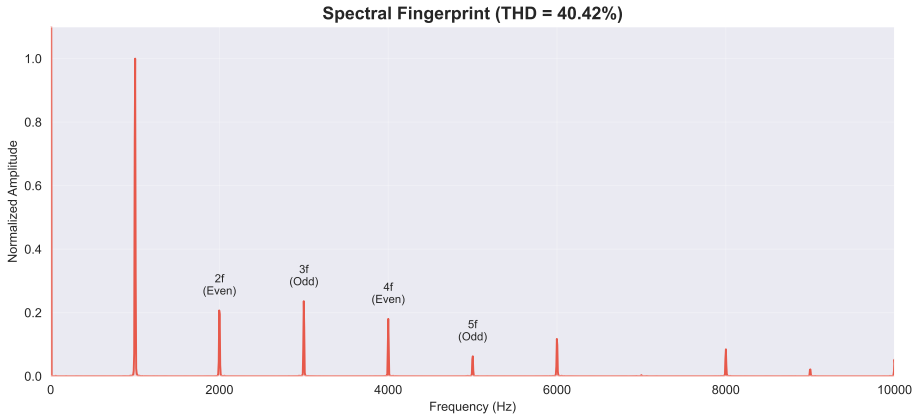


Figure 6.6: **Steady-State Harmonic Fingerprint at 1 kHz.** Burst-mode capture of the DUT driven by a 1 kHz sine. H_2 and H_3 are equal, while the higher even harmonics ($4f$, $6f$, $8f$) outlast the odd ones ($5f$, $7f$). Source: Notebook 04, §6.

6.5.7 Even vs. Odd Harmonic Balance

Comparing H_2 with H_3 alone hides the shape of the full series. Table 6.3 gives the ratio of even-order ($H_2, H_4, \ldots$) to odd-order ($H_3, H_5, \ldots$) harmonic energy over H_2–H_8, computed in Notebook 02, §7 (guitar) and Notebook 04, §6 (1 kHz sine) with a common method (measured fundamental, main-lobe energy, local noise floor removed).

Stimulus	Clean Input	Red Llama Output
1 kHz sine	– (all harmonics < 0.01)	1.34
Guitar low E	1.05	1.88

Table 6.3: Even/odd harmonic energy ratio (H_2–H_8). Values above 1 mean the even series dominates. Source: Notebook 02, §7 and Notebook 04, §6.

With the pure tone, H_2 and H_3 are equal (0.226 vs. 0.230 of the fundamental), but the higher orders split: H_4, H_6 and H_8 (0.173, 0.122, 0.083) outlast H_5 and H_7 (0.065, 0.003). Starting from a clean source, the circuit therefore produces an even-dominant series. With the guitar, whose clean note is already harmonically rich but balanced, the circuit raises H_2 from 1.47 to 2.21 and lowers H_3 from 1.73 to 1.31.

Both stimuli thus confirm the even-order dominance reported in Section 6.4, the signature of asymmetric clipping. The effect is stronger with the guitar, whose input harmonics intermodulate in the nonlinearity and whose clean and Red Llama captures are separate, unmatched plucks.

6.5.8 Limitations

- All DUT measurements were taken at maximum gain; the small-signal response was not measured.
- No data is available below $\approx$ 95 Hz, and no phase response is available, since blocks are not phase-continuous.
- Absolute gain is not reported: the source and DUT sweeps were played at different amplitudes, and playback volume was not logged.

Chapter 7

Conclusion

7.1 Synthesis: The Recursive Engineering Loop

This technical report documented a deviation from standard hobbyist electronics toward a rigorous systems engineering approach. The project verified that the "Full-Stack Gap" described in Chapter 1 can be bridged not by purchasing better tools, but by engineering them.

The development of the four interdependent systems—Logistics, Infrastructure, Device, and Instrumentation—demonstrated that the principles of software architecture (modularity, determinism, and regression testing)[7, 6] are directly transferable to physical fabrication. The success of the **Red Llama** build was not a result of manual dexterity, but of the logistical determinism provided by **Star Ground** and the electrical stability provided by the **Linear Power Regulator**.

7.2 Summary of Empirical Findings

The analytical phase (Chapter 6) provided quantitative validation of the system's performance. The data supports the following conclusions regarding the **Red Llama** topology and the custom **RP2040 DAQ**:

1. **Topological Confirmation (The "Tube" Hypothesis):** The transfer characteristic analysis confirmed that the CD4049 CMOS Hex Inverter, when biased into the linear region, exhibits a continuous hyperbolic tangent (tanh) saturation curve.[16] The absence of derivative discontinuities at the saturation threshold (the "Soft Knee") mathematically distinguishes this circuit from standard silicon diode clipping.

2. **Harmonic Fingerprint:** Spectral decomposition revealed a dominant 2nd-order harmonic ($f = 2f_0$) and a decaying series of even-order harmonics. This confirms that the CMOS inverter mimics the asymmetrical transfer function of single-ended Class A vacuum tube triodes, validating the psychoacoustic description of the device as "warm" or "amp-like."[29] A pure-tone stimulus confirms the even-order dominance: from a clean 1 kHz source the circuit produces 1.34× more even- than odd-harmonic energy (1.88× with the guitar, versus 1.05 for the clean note).

3. **Swept Transfer Function:** At maximum gain the DUT is flat within 0.5 dB from 97 Hz to 3.4 kHz, rolling off gently to -2.8 dB at 19.1 kHz. Its distortion is heavy and frequency-independent ($H_2 \approx H_3 \approx -13$ dBc), more than 30 dB above the measurement chain's floor, and the large-signal transfer curve is an asymmetric sigmoid with capacitive memory.

4. **Instrumentation Efficacy:** The custom RP2040 Data Acquisition System met or exceeded all design requirements.

- **Bandwidth:** A sample rate of 97.69 kSps ± 0.01% was verified against crystal-referenced tones, providing a Nyquist frequency of 48.8 kHz, well beyond the audible spectrum.[30]
- **Fidelity:** The system demonstrated a read noise floor of 1.29mV$_{rms}$ and a full-scale dynamic range of 59.1 dB (9.5 effective bits), proving it is a viable tool for audio frequency analysis.

7.3 Phase II: Active Signal Injection

Phase II replaced passive instrument capture with active signal injection, using the host laptop's audio DAC as a signal generator.

- **Hardware Interface:** A custom impedance-matching cable (Aux TRS → Mono TS) bridges the consumer line-level output of the host to the high-impedance instrument input of the DUT.
- **Software Generation:** A Python-based Direct Digital Synthesis (DDS) engine generates logarithmic sine sweeps (20 Hz–20 kHz) and steady-state test tones.
- **Analysis:** Chunked harmonic fitting (Section 6.5) recovers the frequency response and harmonic distortion versus frequency from stream-mode captures, despite the untimed gaps between blocks.

Phase II also exposed the main limitation of the v0 prototype: stream mode cannot record a phase-continuous signal longer than one burst. This rules out phase measurement and ESS deconvolution, and directly motivates the continuous-acquisition architecture proposed below.

7.4 Future Roadmap: The Silicon Revision (v1.0)

The current device, constructed on perfboard with discrete modules, serves as a validated prototype (v0). While effective for characterizing soft-clipping topologies, the reliance on the RP2040's internal 12-bit ADC and manual jumper configuration limits its utility as a precision laboratory standard.

Based on the empirical data collected in this report, a comprehensive "Silicon Revision" (v1.0) is proposed as the first product iteration, transitioning the system from a development tool to an autonomous measurement engine.

7.4.1 Hardware Architecture

The next iteration will migrate from modular sub-assemblies to a unified **4-layer PCB** utilizing Surface Mount Devices (SMD).

- **Noise Floor:** A dedicated internal ground plane will replace the star-ground wiring to further suppress Electromagnetic Interference (EMI), with a target noise floor below −80dB.
- **Connectivity:** Standard 0.1" headers will be replaced with shielded **BNC connectors** to interface with standard oscilloscope probes.
- **Continuous Acquisition:** DMA-driven, double-buffered sampling will stream data without the inter-block gaps of the prototype's request–response protocol (Section 6.5), enabling phase-continuous captures of arbitrary length.

7.4.2 Signal Chain Evolution

To achieve *Audio Analyzer* grade performance, the internal data converters will be bypassed in favor of dedicated I2S codecs:

- **Acquisition (ADC):** An external 24-bit I2S ADC (e.g., PCM1808) will replace the internal 12-bit converter. This will increase the dynamic range significantly, allowing for the resolution of harmonic distortion products currently buried in the quantization noise.
- **Generation (DAC):** A high-resolution DAC (e.g., PCM5102) will be integrated directly onto the bus. This eliminates the dependency on the host laptop's sound card (described in Section 7.3) and allows the MCU to generate perfectly synchronized test tones for automated Bode plotting.

7.4.3 Programmable Analog Front End (AFE)

The current manual configuration (physical jumpers for Gain and AC/DC coupling) introduces mechanical wear and prevents remote automation. The v1.0 AFE will utilize:

- **Relay Switching:** To toggle between AC and DC coupling modes without signal degradation.
- **Programmable Gain Amplifier (PGA):** To provide software-controlled attenuation, enabling the system to auto-range between millivolt-level guitar pickups and high-voltage modular synthesizer rails (±12V).

7.5 Closing Statement

The **Systems Audio Lab** project stands as a proof-of-concept for the *systems* approach to maker-electronics. By refusing to treat the supply chain, power delivery, and validation as externalities, the project yielded a device with a verified noise floor and a mathematically characterized distortion profile. The proposed evolution to v1.0 demonstrates that the distinction between software code and silicon hardware is increasingly arbitrary; both are simply distinct layers of the same engineering stack.

Appendix A

Source Code

A.1 Firmware: Acquisition Loop

MicroPython logic running on the RP2040, handling the high-speed burst capture.

```
# pyright: reportMissingImports=false

import array
import gc
import sys

import machine
import micropython
import uselect

# Configuration
ADC_PIN_NUM = 28
MAX_SAMPLES = 16384  # Science Mode Buffer
LIVE_SAMPLES = 1024  # Video Mode Buffer

# Setup ADC
adc = machine.ADC(machine.Pin(ADC_PIN_NUM))
# Allocate max memory once to avoid fragmentation
adc_buffer = array.array("H", [0] * MAX_SAMPLES)


# Pre-compile the capture function to arm machine code
@micropython.native
def capture_burst(adc_obj, buf, size: int):
    """
    Read a burst of analog values into a buffer using native code generation.

    This function is decorated with @micropython.native for speed. It avoids
    memory allocation during the loop to maintain deterministic timing.

    Parameters
    ----------
    adc_obj : machine.ADC
        The configured ADC instance to read from.
    buf : array.array
        The pre-allocated buffer (array of unsigned short 'H') to store data.
    size : int
        The number of samples to capture. Must be <= len(buf).
    """
    for i in range(size):
        buf[i] = adc_obj.read_u16()
```

```

def main():
    """
    Execute the main firmware loop.

    Listens for single-character commands over USB Serial (stdin):
    - 's': Science Mode (High Res). Captures MAX_SAMPLES, with GC disabled
           during capture for stability. Sends full buffer.
    - 'v': Video Mode (Low Latency). Captures LIVE_SAMPLES. Sends partial
           buffer immediately. No GC manipulation for higher frame rates.
    """
    poll_obj = uselect.poll()
    poll_obj.register(sys.stdin, uselect.POLLIN)

    while True:
        # Poll with a 100ms timeout to allow other background tasks if necessary
        if not poll_obj.poll(100):
            continue

        # Read 1 byte from stdin
        cmd = sys.stdin.read(1)

        # 's' = SCIENCE MODE (High Res, Deep Buffer)
        if cmd == "s":
            gc.disable()
            capture_burst(adc, adc_buffer, MAX_SAMPLES)
            gc.enable()
            # Send FULL buffer
            sys.stdout.buffer.write(memoryview(adc_buffer))
            gc.collect()

        # 'v' = VIDEO MODE (Low Latency, Short Buffer)
        elif cmd == "v":
            # No need to disable GC for short bursts, keeps it snappy
            capture_burst(adc, adc_buffer, LIVE_SAMPLES)

            # Send ONLY the first 1024 samples
            # This slicing [:LIVE_SAMPLES] is very fast (no copying)
            sys.stdout.buffer.write(memoryview(adc_buffer)[:LIVE_SAMPLES])

            # Note: We don't GC here to keep frame rate high


if __name__ == "__main__":
    main()
```

Listing A.1: main.py

A.2 Host: Calibration Algorithm

The weighted-average logic used to derive the true sample rate from mains hum (Ref. Chapter 5).

```
import json
import os
import time

import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import windows

from . import audio, config, daq, dsp
```

```python


def save_calibration(fs: float, method: str = "mains_60hz") -> None:
    """
    Save the calibrated sampling rate to the calibration JSON file.

    Creates the parent directory for the calibration file if it does not exist.
    The file will contain the sampling rate, a timestamp, and the hardware default.

    Parameters
    ----------
    fs : float
        The calculated true sampling rate in Hz.
    method : str, optional
        How `fs` was derived (e.g. "mains_60hz", "reference_tone_1000hz").
    """
    # 1. Create data directory if it doesn't exist yet
    os.makedirs(os.path.dirname(config.CALIBRATION_FILE_PATH), exist_ok=True)

    data = {
        "fs": fs,
        "timestamp": time.time(),
        "date_str": time.strftime("%Y-%m-%d %H:%M:%S"),
        "hardware_default": config.FS_DEFAULT,
        "method": method,
    }

    with open(config.CALIBRATION_FILE_PATH, "w") as f:
        json.dump(data, f, indent=4)

    print(f"💾 Calibration saved to {config.CALIBRATION_FILE_PATH}")


def load_calibration() -> float | None:
    """
    Load a previously saved sampling rate from disk if available.

    Returns
    -------
    Optional[float]
        The cached sampling rate in Hz if the file exists and is valid.
        Returns None if the file is missing or corrupted.
    """
    if not os.path.exists(config.CALIBRATION_FILE_PATH):
        return None

    try:
        with open(config.CALIBRATION_FILE_PATH) as f:
            data = json.load(f)

        fs = data.get("fs")
        date = data.get("unknown date")
        print(f"📂 Loaded cached calibration: {fs:.1f} Hz ({date})")
        return float(fs)

    except Exception as e:
        print(f"⚠️ Could not load calibration: {e}")
        return None


def calibrate_fs_robust(visualize: bool = True) -> float:
    """
    Calculate the true sampling rate by analyzing 60Hz mains hum.
```

```
    Mains hum is a weak reference: the grid wanders by a few hundredths of a
    hertz and a 16k-sample burst holds only ~10 cycles at low SNR. Prefer
    `calibrate_fs_with_tone` when a sound card is available.

    This function captures a burst of data, identifying the mains frequency
    assuming the user is touching the input jack. It uses a weighted average
    of FFT bins around the peak for sub-bin precision.

    Parameters
    ----------
    visualize : bool, optional

        If True, plots the first few cycles of the
        captured signal for visual verification.

        Default is True.

    Returns
    -------
    float
        The calculated real sampling rate in Hz.
        Returns config.FS_DEFAULT if calibration fails or SNR is too low.
    """
    print("👉 TOUCH JACK TIP NOW for 60Hz Calibration...")

    try:
        # 1. Capture
        with daq.DAQInterface() as device:
            raw = device.capture_burst()
            voltages = dsp.raw_to_volts(raw)

        # 2. AC Couple
        ac_signal = dsp.remove_dc(voltages)

        # 3. FFT Analysis
        n = len(ac_signal)
        windowed = ac_signal * windows.hann(n)
        fft_mag = np.abs(np.fft.fft(windowed)[: n // 2])

        # 4. Find Peak
        peak_idx = int(np.argmax(fft_mag))
        peak_mag = fft_mag[peak_idx]

        # 5. Calculate SNR (exclude 10 bins around peak)
        noise_mask = np.ones(len(fft_mag), dtype=bool)
        noise_mask[max(0, peak_idx - 5) : min(len(fft_mag), peak_idx + 5)] = False
        noise_floor = np.mean(fft_mag[noise_mask])

        # Peak bin vs. mean of all other bins. This includes the FFT's
        # processing gain (~39 dB for 16k points), so it is NOT a time-domain
        # SNR; it only checks that a clear 60 Hz line is present.
        peak_ratio = peak_mag / noise_floor if noise_floor > 0 else 0

        print("\n--- DIAGNOSTICS ---")
        print(f"Spectral peak-to-floor ratio: {peak_ratio:.1f}x (not SNR)")

        if peak_ratio < 10:
            print("❌ WARNING: Signal too weak! Touch the jack firmly.")
            return float(config.FS_DEFAULT)

        # 6. Weighted Average for Precision
        win_idxs = np.arange(peak_idx - 2, peak_idx + 3)
```

```
        # Boundary check
        valid_idxs = win_idxs[(win_idxs >= 0) & (win_idxs < len(fft_mag))]

        precise_idx = np.sum(valid_idxs * fft_mag[valid_idxs]) / np.sum(
            fft_mag[valid_idxs]
        )

        # 7. Calculate Real FS: Fs = (Target_Freq * N) / Peak_Index
        real_fs = (60.0 * n) / precise_idx
        print(f"Calculated FS:       {real_fs:.1f} Hz")

        if visualize:
            plt.figure(figsize=(10, 3))
            samples_per_cycle = int(real_fs / 60)
            plt.plot(ac_signal[: samples_per_cycle * 5], color="cyan")
            plt.title("Visual Check: Clean 60Hz Sine Waves?")
            plt.grid(True, alpha=0.3)
            plt.show()

        return float(real_fs)

    except Exception as e:
        print(f"❌ Calibration Failed: {e}")
        return float(config.FS_DEFAULT)


def fs_from_reference_tone(
    signal_arr: np.ndarray,
    fs_assumed: float,
    f_ref: float,
    n_harmonics: int = 1,
) -> tuple[float, float]:
    """
    Derive the true sampling rate from a capture of a known reference tone.

    If a tone of true frequency `f_ref` measures `f_meas` under an assumed
    rate, the true rate is ``fs_assumed * f_ref / f_meas``. A sound card's
    crystal clock (typically +/-50 ppm) makes its tones a far better reference
    than mains hum.

    Parameters
    ----------
    signal_arr : np.ndarray
        Burst capture of the reference tone.
    fs_assumed : float
        Sampling rate that was assumed when the capture was made.
    f_ref : float
        True frequency of the reference tone in Hz.
    n_harmonics : int, optional
        Harmonics included in the frequency fit (use > 1 for non-sine tones).

    Returns
    -------
    Tuple[float, float]
        (fs_true, std) in Hz, where std propagates the frequency fit's bound.
    """
    f_meas, f_std = dsp.estimate_tone_frequency(
        signal_arr, fs_assumed, f_ref, n_harmonics
    )
    fs_true = fs_assumed * f_ref / f_meas
    return float(fs_true), float(fs_true * f_std / f_meas)


```

```
def calibrate_fs_with_tone(
    f_ref: float = 1000.0, amp: float = 0.5, settle_s: float = 0.5
) -> float:
    """
    Calibrate the sampling rate by capturing a sound-card reference tone.

    Plays a sine of `f_ref` through the host audio output (loop it back into
    the DAQ input), captures a burst, and saves the derived rate.

    Parameters
    ----------
    f_ref : float, optional
        Reference tone frequency in Hz.
    amp : float, optional
        Digital amplitude of the tone (0.0 to 1.0).
    settle_s : float, optional
        Seconds to let playback settle before capturing.

    Returns
    -------
    float
        The calibrated sampling rate in Hz.
    """
    with audio.ContinuousOscillator("sine", f_ref, amp):
        time.sleep(settle_s)
        with daq.DAQInterface() as device:
            volts = dsp.raw_to_volts(device.capture_burst())

    fs_true, fs_std = fs_from_reference_tone(volts, config.FS_DEFAULT, f_ref)
    print(f"Calibrated FS: {fs_true:.1f} +/- {fs_std:.1f} Hz (ref {f_ref:.0f} Hz)")
    save_calibration(fs_true, method=f"reference_tone_{f_ref:.0f}hz")
    return fs_true
```

Listing A.2: src/calibration.py

A.3 Host: Signal Processing (DSP)

FFT windowing and THD calculation routines (Ref. Chapter 6).

```
import numpy as np
import scipy.signal as spsig
from scipy.optimize import minimize_scalar

from . import config


def raw_to_volts(raw_data: np.ndarray) -> np.ndarray:
    """
    Convert raw uint16 ADC values to float64 voltages.

    Parameters
    ----------
    raw_data : np.ndarray
        Array of raw ADC integer values (0 to config.ADC_MAX_VAL).

    Returns
    -------
    np.ndarray
        Array of voltage values scaled to config.V_REF.
    """
    return (raw_data / config.ADC_MAX_VAL) * config.V_REF

```

```python

def remove_dc(signal: np.ndarray) -> np.ndarray:
    """
    Subtract the mean (DC offset) from the signal.

    Parameters
    ----------
    signal : np.ndarray
        Input signal array.

    Returns
    -------
    np.ndarray
        The signal centered around 0.0.
    """
    return signal - np.mean(signal)  # type: ignore[no-any-return]


def software_trigger(signal: np.ndarray, threshold: float = config.V_MID) -> np.ndarray:
    """
    Stabilize a periodic waveform by aligning the first rising edge.

    Rolls the array to align the first rising edge crossing with index 0.

    Parameters
    ----------
    signal : np.ndarray
        Input signal array.
    threshold : float, optional
        Voltage level to define the trigger crossing. Defaults to config.V_MID.

    Returns
    -------
    np.ndarray
        A cyclically shifted copy of the signal starting at the first rising edge.
        Returns the original signal if no crossing is found.
    """
    # Boolean mask: (Current < Thresh) AND (Next >= Thresh)
    crossings = np.where((signal[:-1] < threshold) & (signal[1:] >= threshold))[0]

    if crossings.size > 0:
        return np.roll(signal, -crossings[0])
    return signal


def compute_spectrum(signal: np.ndarray, fs: float) -> tuple[np.ndarray, np.ndarray]:
    """
    Compute the one-sided FFT magnitude spectrum using a Hann window.

    Parameters
    ----------
    signal : np.ndarray
        (N,) Input time-domain signal.
    fs : float
        Sampling rate in Hz.

    Returns
    -------
    freq_axis : np.ndarray
        (N/2 + 1,) Array of frequency bins in Hz.
    fft_mag : np.ndarray
        (N/2 + 1,) Array of magnitude values (normalized).
    """
```

```
    n = signal.size
    # Apply window to reduce spectral leakage
    windowed = signal * spsig.windows.hann(n)

    # FFT
    fft_complex = np.fft.rfft(windowed)
    fft_mag = (2.0 / n) * np.abs(fft_complex)

    # Frequency axis
    freq_axis = np.fft.rfftfreq(n, d=1.0 / fs)

    return freq_axis, fft_mag


def estimate_fundamental(
    freqs: np.ndarray, mags: np.ndarray, fmin: float = 20.0, fmax: float = 2000.0
) -> float:
    """
    Find the frequency of the dominant peak within a specific band.

    Parameters
    ----------
    freqs : np.ndarray
        Array of frequency bins.
    mags : np.ndarray
        Array of magnitudes corresponding to freqs.
    fmin : float, optional
        Minimum frequency to consider. Defaults to 20.0.
    fmax : float, optional
        Maximum frequency to consider. Defaults to 2000.0.

    Returns
    -------
    float
        The frequency in Hz of the largest peak within [fmin, fmax].
        Returns 0.0 if no peak is found or arrays are empty.
    """
    if freqs.size == 0:
        return 0.0

    mask = (freqs >= fmin) & (freqs <= fmax)
    if not np.any(mask):
        return 0.0

    f_subset = freqs[mask]
    m_subset = mags[mask]

    # Simple peak finding
    idx = np.argmax(m_subset)
    return float(f_subset[idx])


def _component_power(power: np.ndarray, df: float, f_target: float) -> float:
    """
    Noise-corrected energy of the spectral peak nearest `f_target`.

    Searches +/-3 bins for the peak, sums power over the Hann main lobe
    (+/-2 bins) so the result does not depend on where the tone falls between
    bins, and subtracts the local floor estimated 4-10 bins either side.
    """
    lobe, search, ring_lo, ring_hi = 2, 3, 4, 10
    k0 = round(f_target / df)
    lo, hi = max(k0 - search, 0), min(k0 + search + 1, len(power))
```

```
    if lo >= hi:
        return 0.0
    k = lo + int(np.argmax(power[lo:hi]))

    peak = power[max(k - lobe, 0) : k + lobe + 1]
    ring = np.concatenate(
        [
            power[max(k - ring_hi, 0) : max(k - ring_lo + 1, 0)],
            power[k + ring_lo : k + ring_hi + 1],
        ]
    )
    floor = float(np.mean(ring)) if ring.size else 0.0
    return max(0.0, float(np.sum(peak)) - floor * peak.size)


def harmonic_amplitudes(
    signal_arr: np.ndarray,
    fs: float,
    fundamental_freq: float,
    n_harmonics: int = 10,
) -> tuple[float, np.ndarray]:
    """
    Measure the fundamental and the amplitudes of its harmonics.

    The true fundamental is located within +/-10% of `fundamental_freq` (real
    tones drift from nominal, e.g. a sharp string or an uncalibrated sound
    card) and refined by parabolic interpolation. Each harmonic's energy is
    summed over its Hann main lobe with the local noise floor removed.

    Parameters
    ----------
    signal_arr : np.ndarray
        Time-domain signal array.
    fs : float
        Sampling rate in Hz.
    fundamental_freq : float
        Nominal fundamental frequency in Hz.
    n_harmonics : int, optional
        Number of harmonics to measure, including the fundamental.

    Returns
    -------
    Tuple[float, np.ndarray]
        (measured_fundamental_hz, amplitudes) where amplitudes is
        (n_harmonics,) relative to the fundamental (index 0 = 1.0). Harmonics
        above Nyquist are 0. Returns (0.0, zeros) if no fundamental is found.
    """
    freqs, mags = compute_spectrum(signal_arr, fs)
    df = freqs[1] - freqs[0]
    power = mags**2
    empty = (0.0, np.zeros(n_harmonics))

    band = (freqs > 0.9 * fundamental_freq) & (freqs < 1.1 * fundamental_freq)
    if not np.any(band):
        return empty
    k1 = int(np.flatnonzero(band)[np.argmax(mags[band])])
    if 0 < k1 < len(mags) - 1:
        a, b, c = np.log(mags[k1 - 1 : k1 + 2] + 1e-20)
        k1_frac = k1 + 0.5 * (a - c) / (a - 2 * b + c)
    else:
        k1_frac = float(k1)
    f1 = float(k1_frac * df)

```

```
    powers = np.array(
        [
            _component_power(power, df, h * f1) if h * f1 < freqs[-1] else 0.0
            for h in range(1, n_harmonics + 1)
        ]
    )
    if powers[0] <= 0:
        return empty

    return f1, np.sqrt(powers / powers[0])


def estimate_tone_frequency(
    signal_arr: np.ndarray,
    fs: float,
    fundamental_freq: float,
    n_harmonics: int = 1,
) -> tuple[float, float]:
    """
    Estimate a steady tone's frequency to well below one FFT bin.

    Starts from the interpolated FFT peak, then minimises the residual of a
    least-squares fit of the fundamental plus `n_harmonics - 1` harmonics over
    a +/-1 bin window. For a stable, pure tone this resolves frequency far
    more finely than the bin width, limited by noise rather than by N.

    Parameters
    ----------
    signal_arr : np.ndarray
        Time-domain signal array.
    fs : float
        Sampling rate in Hz (assumed).
    fundamental_freq : float
        Nominal tone frequency in Hz; the peak within +/-10% is used.
    n_harmonics : int, optional
        Harmonics included in the fit model (use > 1 for distorted tones).

    Returns
    -------
    Tuple[float, float]
        (frequency_hz, std_hz) where std_hz is the Cramer-Rao bound implied
        by the fitted amplitude and residual noise.
    """
    x = np.asarray(signal_arr, dtype=np.float64)
    x = x - np.mean(x)
    n = x.size
    t = np.arange(n) / fs

    def fit(f: float) -> tuple[float, float]:
        cols = [np.ones(n)]
        for h in range(1, n_harmonics + 1):
            cols += [np.cos(2 * np.pi * h * f * t), np.sin(2 * np.pi * h * f * t)]
        basis = np.stack(cols, axis=1)
        coeffs, *_ = np.linalg.lstsq(basis, x, rcond=None)
        resid = x - basis @ coeffs
        return float(np.sum(resid**2)), float(np.hypot(coeffs[1], coeffs[2]))

    f0, _ = harmonic_amplitudes(x, fs, fundamental_freq, n_harmonics=1)
    if f0 == 0.0:
        return 0.0, float("inf")
    bin_hz = fs / n
    result = minimize_scalar(
        lambda f: fit(f)[0],
```

```
        bounds=(f0 - bin_hz, f0 + bin_hz),
        method="bounded",
        options={"xatol": 1e-6},
    )
    f_hat = float(result.x)

    ssr, amp = fit(f_hat)
    noise_var = ssr / n
    snr = (amp**2 / 2) / noise_var if noise_var > 0 else np.inf
    duration = n / fs
    std = float(np.sqrt(6 / (n * snr)) / (np.pi * duration))
    return f_hat, std


def even_odd_energy_ratio(amplitudes: np.ndarray) -> float:
    """
    Ratio of even-order to odd-order harmonic energy (fundamental excluded).

    Parameters
    ----------
    amplitudes : np.ndarray
        Harmonic amplitudes, index 0 = fundamental (as from
        `harmonic_amplitudes`).

    Returns
    -------
    float
        sqrt(sum of H2, H4, ... squared) / sqrt(sum of H3, H5, ... squared).
        Above 1 means the even series carries more energy.
    """
    even = float(np.sum(amplitudes[1::2] ** 2))
    odd = float(np.sum(amplitudes[2::2] ** 2))
    return float(np.sqrt(even / odd)) if odd > 0 else float("inf")


def calculate_selective_thd(
    signal_arr: np.ndarray,
    fs: float,
    fundamental_freq: float = 82.4,
    n_harmonics: int = 10,
) -> float:
    """
    Calculate Total Harmonic Distortion (THD) from selective harmonics.

    Sums ONLY the energy at specific harmonic frequencies, ignoring the
    broad-band noise floor. See `harmonic_amplitudes` for how the fundamental
    and harmonics are measured.

    Parameters
    ----------
    signal_arr : np.ndarray
        Time-domain signal array.
    fs : float
        Sampling rate in Hz.
    fundamental_freq : float, optional
        Nominal fundamental frequency (e.g., 82.4 for low E).
    n_harmonics : int, optional
        Number of harmonics to include in the calculation.

    Returns
    -------
    float
        THD percentage (0.0 to 100.0+).
```

```
    """
    _, amps = harmonic_amplitudes(signal_arr, fs, fundamental_freq, n_harmonics)
    return float(100 * np.sqrt(np.sum(amps[1:] ** 2)))


def smart_align(sig_ref: np.ndarray, sig_target: np.ndarray) -> np.ndarray:
    """
    Align 'sig_target' to match the phase of 'sig_ref' using cross-correlation.

    Parameters
    ----------
    sig_ref : np.ndarray
        The reference signal (stationary).
    sig_target : np.ndarray
        The signal to be shifted.

    Returns
    -------
    np.ndarray
        The shifted version of sig_target that aligns best with sig_ref.
    """
    # 1. Compute Cross-Correlation
    correlation = spsig.correlate(sig_target, sig_ref, mode="full")
    lags = spsig.correlation_lags(sig_target.size, sig_ref.size, mode="full")

    # 2. Find the lag that maximizes correlation
    lag = lags[np.argmax(correlation)]

    # 3. Shift the target signal
    return np.roll(sig_target, -lag)
```

Listing A.3: src/dsp.py

A.4 Host: Function Generator Logic (Phase II)

Core logic for the "Laptop-as-Generator" system, used to generate the frequency sweeps analyzed in Section 6.5.

```
"""
Script to play a logarithmic sine sweep and visualize it in real-time.

This script generates a log sine sweep audio buffer, plays it through the system
output, and simultaneously streams data from the DAQ to a live oscilloscope
window. This allows for visual verification of the transfer function measurement
setup before committing to a recording.
"""

import sounddevice as sd
from sysaudio import audio, daq, viz

# Configuration
F_START: float = 20.0  # Start Frequency (Hz)
F_END: float = 20000.0  # End Frequency (Hz)
DURATION: float = 5.0  # Seconds
AMPLITUDE: float = 0.5  # 0.0 to 1.0


def main() -> None:
    """
    Execute the log sweep playback and live capture script.

```

```
    Generates the sweep buffer, establishes the DAQ connection, and launches
    the live visualization. The audio playback is triggered via a callback
    once the visualization window is fully initialized to ensure synchronization.
    """
    print(f"Generating Log Sweep ({F_START:.0f}-{F_END:.0f}Hz)...")

    # 1. Generate Buffer
    fs_audio: int = 48000
    wave = audio.generate_log_sweep(F_START, F_END, DURATION, fs_audio, AMPLITUDE)

    # Define the play function
    def start_playback() -> None:
        print("🔊 Playing Sweep...")
        sd.play(wave, samplerate=fs_audio, blocking=False)

    # 2. Start Scope
    # on_launch triggers playback only when the window is visible
    with daq.DAQInterface() as device:
        viz.run_live_scope(
            device.stream_generator(),
            title="Transfer Function: Log Sine Sweep",
            stop_condition=lambda: not sd.get_stream().active,
            on_launch=start_playback,
        )

    print("Done.")


if __name__ == "__main__":
    main()
```

Listing A.4: scripts/signal/play_sweep.py

A.5 Host: Chunked Sweep Analysis (Phase II)

Per-block harmonic fitting used to recover transfer functions from stream-mode captures (Ref. Section 6.5).

```
"""
Frequency response and harmonic analysis of log sweeps captured in stream mode.

Stream-mode captures are 1024-sample chunks separated by untimed USB dead time
(roughly 25% of wall-clock time is lost). Phase is not continuous across chunk
boundaries, so whole-signal methods like ESS deconvolution do not apply.

Each chunk, however, is gap-free and short enough that the sweep is
quasi-stationary within it. This module:

1. Recovers each chunk's position in the sweep by fitting a constant chunk
   period to the per-chunk dominant frequency.
2. Fits the exact log-sweep phase model (fundamental + harmonics) to each chunk
   by linear least squares.
3. Resamples the per-chunk amplitudes onto a common log-frequency grid so that
   source and DUT captures can be compared.
"""

from dataclasses import dataclass

import numpy as np

from . import config

```

```python

@dataclass(frozen=True)
class SweepHarmonics:
    """
    Per-chunk harmonic amplitudes extracted from a chunked log sweep.

    Attributes
    ----------
    freqs : np.ndarray
        (M,) fundamental frequency at each valid chunk centre in Hz.
    amplitudes : np.ndarray
        (M, K) peak amplitude of harmonics 1..K in input units. NaN where the
        harmonic would exceed the analysis bandwidth.
    residual_rms : np.ndarray
        (M,) RMS of what the harmonic model failed to explain (noise + higher
        harmonics), in input units.
    chunk_period : float
        Fitted wall-clock time between chunk starts in seconds.
    """

    freqs: np.ndarray
    amplitudes: np.ndarray
    residual_rms: np.ndarray
    chunk_period: float


def _sweep_rate(f_start: float, f_end: float, duration: float) -> float:
    """Return the natural log of the per-second frequency ratio of the sweep."""
    return float(np.log(f_end / f_start) / duration)


def _peak_frequencies(
    chunks: np.ndarray, fs: float, n_fft: int = 1 << 15
) -> np.ndarray:
    """
    Estimate the dominant frequency of each chunk.

    Uses a Hann-windowed, zero-padded FFT with parabolic interpolation on the
    log magnitude around the peak bin.
    """
    n = chunks.shape[1]
    spec = np.abs(np.fft.rfft(chunks * np.hanning(n), n_fft, axis=1))
    k = np.clip(spec.argmax(axis=1), 1, spec.shape[1] - 2)
    rows = np.arange(len(k))

    a = np.log(spec[rows, k - 1] + 1e-12)
    b = np.log(spec[rows, k] + 1e-12)
    c = np.log(spec[rows, k + 1] + 1e-12)
    offset = 0.5 * (a - c) / (a - 2 * b + c)

    return (k + offset) * fs / n_fft  # type: ignore[no-any-return]


def estimate_chunk_frequencies(
    chunks: np.ndarray,
    fs: float,
    f_start: float,
    f_end: float,
    duration: float,
    fit_band: tuple[float, float] = (300.0, 15000.0),
) -> tuple[np.ndarray, float]:
    """
    Estimate the sweep frequency at the centre of every chunk.
```

```
    Chunks in `fit_band` with healthy signal level are used to robustly fit
    ``t_k = t0 + k * period``, where ``t_k`` is the sweep time implied by each
    chunk's measured frequency. The fit extrapolates to chunks where a direct
    estimate is unreliable (low frequencies, where a chunk holds few cycles).

    Parameters
    ----------
    chunks : np.ndarray
        (N, chunk_size) DC-removed chunks.
    fs : float
        Sampling rate in Hz.
    f_start, f_end, duration : float
        Parameters of the played log sweep.
    fit_band : tuple of float, optional
        Frequency band whose chunks anchor the timing fit.

    Returns
    -------
    Tuple[np.ndarray, float]
        (frequencies, chunk_period) where frequencies is (N,) in Hz.

    Raises
    ------
    ValueError
        If too few chunks fall in `fit_band` to fit the timing model.
    """
    lnr = _sweep_rate(f_start, f_end, duration)
    measured = _peak_frequencies(chunks, fs)
    idx = np.arange(len(chunks))
    t_implied = np.log(np.maximum(measured, 1e-6) / f_start) / lnr

    rms = chunks.std(axis=1)
    candidates = (
        (measured > fit_band[0])
        & (measured < fit_band[1])
        & (rms > 0.3 * np.median(rms))
    )
    if candidates.sum() < 10:
        raise ValueError("Too few chunks inside fit_band to recover sweep timing.")

    # Iteratively reweighted line fit; rejects chunks with misidentified peaks.
    inliers = candidates.copy()
    for _ in range(5):
        coeffs = np.polyfit(idx[inliers], t_implied[inliers], 1)
        resid = t_implied - np.polyval(coeffs, idx)
        spread = 1.5 * np.median(np.abs(resid[inliers]))
        inliers = candidates & (np.abs(resid) < max(3 * spread, 1e-4))

    predicted = f_start * np.exp(lnr * np.polyval(coeffs, idx))

    # Above fit_band[0] a chunk holds enough cycles that its own estimate beats
    # the timing model, which carries per-chunk USB jitter.
    trust_measured = (predicted > fit_band[0]) & (
        np.abs(measured / predicted - 1) < 0.05
    )
    freqs = np.where(trust_measured, measured, predicted)

    return freqs, float(coeffs[0])


def _sweep_basis(n: int, fs: float, f_centre: float, lnr: float, k: int) -> np.ndarray:
    """Build the [DC, cos(phi), sin(phi), ..., cos(k*phi), sin(k*phi)] matrix."""
```

```
    tau = (np.arange(n) - (n - 1) / 2) / fs
    phase = 2 * np.pi * f_centre * np.expm1(lnr * tau) / lnr
    cols = [np.ones(n)]
    for h in range(1, k + 1):
        cols += [np.cos(h * phase), np.sin(h * phase)]
    return np.stack(cols, axis=1)


def fit_chunk_harmonics(
    chunk: np.ndarray,
    fs: float,
    f_centre: float,
    lnr: float,
    n_harmonics: int,
    search: float = 0.03,
    n_search: int = 61,
) -> tuple[np.ndarray, float, float]:
    """
    Fit the log-sweep harmonic model to a single gap-free chunk.

    The centre frequency is first refined by minimising the fundamental-only
    residual over ``f_centre * (1 +/- search)``, then all harmonics are fitted
    jointly.

    Parameters
    ----------
    chunk : np.ndarray
        (n,) samples of one chunk.
    fs : float
        Sampling rate in Hz.
    f_centre : float
        Initial estimate of the fundamental at the chunk centre in Hz.
    lnr : float
        Natural log of the sweep's per-second frequency ratio.
    n_harmonics : int
        Number of harmonics (including the fundamental) to fit.
    search : float, optional
        Fractional half-width of the frequency refinement search.
    n_search : int, optional
        Number of candidate frequencies in the refinement search.

    Returns
    -------
    Tuple[np.ndarray, float, float]
        (amplitudes, refined_f_centre, residual_rms).
    """
    n = len(chunk)
    # Minimise residual rather than maximise amplitude: with only a few cycles
    # per chunk the basis is far from orthogonal, and the fitted amplitude can
    # inflate at the wrong frequency.
    best_f, best_ssr = f_centre, np.inf
    for f in f_centre * np.linspace(1 - search, 1 + search, n_search):
        _, ssr, *_ = np.linalg.lstsq(_sweep_basis(n, fs, f, lnr, 1), chunk, rcond=None)
        if ssr[0] < best_ssr:
            best_f, best_ssr = float(f), float(ssr[0])

    basis = _sweep_basis(n, fs, best_f, lnr, n_harmonics)
    coeffs, *_ = np.linalg.lstsq(basis, chunk, rcond=None)
    amps = np.hypot(coeffs[1::2], coeffs[2::2])
    resid = float(np.std(chunk - basis @ coeffs))

    return amps, best_f, resid

```

```
214
215 def analyze_chunked_sweep(
216     signal: np.ndarray,
217     fs: float,
218     f_start: float = 20.0,
219     f_end: float = 20000.0,
220     duration: float = 5.0,
221     chunk_size: int = config.LIVE_SAMPLES,
222     n_harmonics: int = 5,
223     max_harmonic_freq: float | None = None,
224 ) -> SweepHarmonics:
225     """
226     Extract fundamental and harmonic amplitudes from a stream-mode sweep.
227
228     Parameters
229     ----------
230     signal : np.ndarray
231         Concatenated stream-mode capture (raw ADC counts or volts).
232     fs : float
233         Sampling rate in Hz.
234     f_start, f_end, duration : float, optional
235         Parameters of the played log sweep.
236     chunk_size : int, optional
237         Samples per gap-free chunk (the firmware's stream block size).
238     n_harmonics : int, optional
239         Number of harmonics (including the fundamental) to report.
240     max_harmonic_freq : float, optional
241         Harmonics above this frequency are not fitted. Defaults to 0.4 * fs.
242
243     Returns
244     -------
245     SweepHarmonics
246         Per-chunk results for chunks that hold at least one full cycle and lie
247         inside the swept band.
248     """
249     if max_harmonic_freq is None:
250         max_harmonic_freq = 0.4 * fs
251
252     n_chunks = len(signal) // chunk_size
253     chunks = signal[: n_chunks * chunk_size].astype(np.float64)
254     chunks = chunks.reshape(n_chunks, chunk_size)
255     chunks -= chunks.mean(axis=1, keepdims=True)
256
257     lnr = _sweep_rate(f_start, f_end, duration)
258     f_est, period = estimate_chunk_frequencies(chunks, fs, f_start, f_end, duration)
259
260     freqs, amps, resids = [], [], []
261     for chunk, f0 in zip(chunks, f_est, strict=True):
262         # Below one cycle per chunk the fit is ill-conditioned against DC.
263         if f0 < fs / chunk_size or f0 > f_end:
264             continue
265         k = int(np.clip(max_harmonic_freq // f0, 1, n_harmonics))
266         a, f_ref, resid = fit_chunk_harmonics(chunk, fs, f0, lnr, k)
267         freqs.append(f_ref)
268         amps.append(np.pad(a, (0, n_harmonics - k), constant_values=np.nan))
269         resids.append(resid)
270
271     order = np.argsort(freqs)
272     return SweepHarmonics(
273         freqs=np.asarray(freqs)[order],
274         amplitudes=np.asarray(amps)[order],
275         residual_rms=np.asarray(resids)[order],
276         chunk_period=period,
```

```
    )


def _resample_log(
    freqs: np.ndarray, values: np.ndarray, grid: np.ndarray, smooth: int
) -> np.ndarray:
    """Median-smooth `values` over `smooth` chunks, then interpolate in log-f."""
    valid = np.isfinite(values)
    f, v = freqs[valid], values[valid]
    if len(v) < 2:
        return np.full_like(grid, np.nan)

    half = smooth // 2
    v_smooth = np.array(
        [np.median(v[max(0, i - half) : i + half + 1]) for i in range(len(v))]
    )
    return np.interp(np.log(grid), np.log(f), v_smooth, left=np.nan, right=np.nan)  # type:
    ignore[no-any-return]


def compute_chunked_bode(
    sig_src: np.ndarray,
    sig_dut: np.ndarray,
    fs: float,
    f_start: float = 20.0,
    f_end: float = 20000.0,
    duration: float = 5.0,
    n_harmonics: int = 5,
    n_points: int = 200,
    smooth: int = 5,
) -> dict[str, np.ndarray]:
    """
    Compare source and DUT stream-mode sweeps on a common frequency grid.

    The DUT/source ratio of fundamental amplitudes cancels the response of the
    playback chain and the DAQ front end, leaving the DUT's own response.

    Parameters
    ----------
    sig_src : np.ndarray
        Sweep captured directly from the playback source (loopback).
    sig_dut : np.ndarray
        Sweep captured at the DUT output.
    fs : float
        Sampling rate in Hz.
    f_start, f_end, duration : float, optional
        Parameters of the played log sweep.
    n_harmonics : int, optional
        Number of harmonics (including the fundamental) to analyse.
    n_points : int, optional
        Number of log-spaced output frequencies.
    smooth : int, optional
        Running-median length (in chunks) applied before resampling.

    Returns
    -------
    Dict[str, np.ndarray]
        ``freqs`` (n_points,); ``gain_db`` (n_points,) normalised to 0 dB peak;
        ``dut_harmonics_dbc`` and ``src_harmonics_dbc`` (n_points, n_harmonics - 1)
        giving harmonics 2..K relative to each capture's own fundamental.
    """
    src = analyze_chunked_sweep(
        sig_src, fs, f_start, f_end, duration, n_harmonics=n_harmonics
```

```
    )
    dut = analyze_chunked_sweep(
        sig_dut, fs, f_start, f_end, duration, n_harmonics=n_harmonics
    )

    lo = max(src.freqs[0], dut.freqs[0])
    hi = min(src.freqs[-1], dut.freqs[-1])
    grid = np.geomspace(lo, hi, n_points)

    def dbc(result: SweepHarmonics) -> np.ndarray:
        rel = 20 * np.log10(result.amplitudes[:, 1:] / result.amplitudes[:, :1])
        return np.stack(
            [_resample_log(result.freqs, col, grid, smooth) for col in rel.T], axis=1
        )

    fund_src = _resample_log(src.freqs, src.amplitudes[:, 0], grid, smooth)
    fund_dut = _resample_log(dut.freqs, dut.amplitudes[:, 0], grid, smooth)
    gain_db = 20 * np.log10(fund_dut / fund_src)

    return {
        "freqs": grid,
        "gain_db": gain_db - np.nanmax(gain_db),
        "dut_harmonics_dbc": dbc(dut),
        "src_harmonics_dbc": dbc(src),
    }
```

Listing A.5: sysaudio/chunked_sweep.py

References

[1] *AN-1950: Silently Powering Low Noise Applications*. Texas Instruments. URL: https://www.ti.com/lit/an/snoa543a/snoa543a.pdf (visited on 01/26/2026).

[2] *BOM (Bill of Materials) Pitfalls: 10 Mistakes That Kill Product Launches*. OpenBOM. Aug. 21, 2025. URL: https://www.openbom.com/blog/bom-bill-of-materials-pitfalls-10-mistakes-that-kill-product-launches (visited on 01/26/2026).

[3] *Should You Use Star Point Grounding in a PCB?* NW Engineering LLC. Sept. 26, 2021. URL: https://www.nwengineeringllc.com/article/should-you-use-star-point-grounding-in-a-pcb.php (visited on 01/26/2026).

[4] *A Survey on Hallucination in Large Language Models*. 2023. URL: https://arxiv.org/abs/2311.05232 (visited on 01/26/2026).

[5] jsvine and contributors. *pdfplumber: Plumb a PDF for detailed information about each character, rectangle, and line; extract text and tables*. URL: https://github.com/jsvine/pdfplumber (visited on 01/26/2026).

[6] Holger Krekel et al. *pytest x.y*. Version x.y. Contributors: Holger Krekel and Bruno Oliveira and Ronny Pfannschmidt and Floris Bruynooghe and Brianna Laugher and Florian Bruhin and others. 2004. URL: https://github.com/pytest-dev/pytest.

[7] *pytest-regtest documentation*. https://pytest-regtest.readthedocs.io. Accessed 2026-01-25.

[8] *TL07xx Low-Noise FET-Input Operational Amplifiers (datasheet)*. Texas Instruments. URL: https://www.ti.com/lit/ds/symlink/tl072.pdf (visited on 01/26/2026).

[9] *OPAx134 High-Performance, SoundPlus™ Audio Operational Amplifiers (datasheet)*. Texas Instruments. URL: https://www.ti.com/lit/ds/symlink/opa2134.pdf (visited on 01/26/2026).

[10] *PNP "Positive Ground" Pedal Considerations*. Amplified Parts. URL: https://www.amplifiedparts.com/tech-articles/pnp-positive-ground-pedals (visited on 01/26/2026).

[11] *You can Build the Perfect Germanium Fuzz Face*. ElectroSmash. May 31, 2012. URL: https://www.electrosmash.com/germanium-fuzz (visited on 01/26/2026).

[12] *A Brief Hobbyist Primer on Clipping Diodes*. GuitarPedalX. Aug. 20, 2023. URL: https://www.guitarpedalx.com/news/news/a-brief-hobbyist-primer-on-clipping-diodes (visited on 01/26/2026).

[13] *Datasheet for Tayda part A-1597*. Tayda Electronics. URL: https://www.taydaelectronics.com/datasheets/files/A-1597.PDF (visited on 01/26/2026).

[14] *1N5817: Schottky Barrier Rectifier Diode (datasheet)*. Distributor-hosted datasheet. Tayda Electronics. URL: https://www.taydaelectronics.com/datasheets/A-159.pdf (visited on 01/26/2026).

[15] EIC. *1N4001 Standard Recovery Power Rectifier, 1 A, 50 V (DO-41) Datasheet*. A-162. EIC. URL: https://www.taydaelectronics.com/datasheets/A-162.pdf.

[16] *CD4049UB, CD4050B: CMOS Hex Buffer/Converter (datasheet)*. Texas Instruments. URL: https://www.ti.com/lit/ds/symlink/cd4049ub.pdf (visited on 01/26/2026).

[17] *Tayda Red Llama Overdrive*. https://www.taydakits.com/instructions/red-llama. Tayda Electronics.

[18] *RP2040 Datasheet (RP-008371-DS)*. Raspberry Pi Ltd. URL: https://pip-assets.raspberrypi.com/categories/814-rp2040/documents/RP-008371-DS-1-rp2040-datasheet.pdf?disposition=inline (visited on 01/26/2026).

[19] *RP2040 Datasheet*. Raspberry Pi Ltd. URL: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf (visited on 01/26/2026).

[20] *BAT54: Schottky barrier diode (datasheet)*. Nexperia. July 1, 2022. URL: https://assets.nexperia.com/documents/data-sheet/BAT54.pdf (visited on 01/26/2026).

[21] *1N4148 Switching Diode (datasheet)*. Generic / distributor copy. URL: https://www.gotronic.fr/pj2-1n4148-1661.pdf (visited on 01/26/2026).

[22] *Maximising MicroPython speed*. MicroPython Project. URL: https://docs.micropython.org/en/latest/reference/speed_python.html (visited on 01/26/2026).

[23] *Signal Processing (scipy.signal) — SciPy Tutorial*. SciPy. URL: https://docs.scipy.org/doc/scipy/tutorial/signal.html (visited on 01/26/2026).

[24] *numpy.fft.fft — NumPy Manual*. NumPy. URL: https://numpy.org/doc/stable/reference/generated/numpy.fft.fft.html (visited on 01/26/2026).

[25] *Fourier Transforms (scipy.fft) — SciPy Tutorial*. SciPy. URL: https://docs.scipy.org/doc/scipy/tutorial/fft.html (visited on 01/26/2026).

[26] *scipy.signal.windows.hann — SciPy Manual*. SciPy. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.windows.hann.html (visited on 01/26/2026).

[27] Fredric J. Harris. "On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform". In: *Proceedings of the IEEE* 66.1 (1978), pp. 51–83. URL: https://ece.uprm.edu/~domingo/teaching/inel5309/On%20the%20Use%20of%20Windows%20for%20Harmonic%20Analysis%20with%20the%20DFT.pdf (visited on 01/26/2026).

[28] Angelo Farina. "Simultaneous Measurement of Impulse Response and Distortion with a Swept-Sine Technique". In: *Audio Engineering Society Convention 108*. Paper 5093. Paris, France, 2000.

[29] *When Distortion Is Good*. Sonarworks. URL: https://www.sonarworks.com/blog/learn/when-distortion-is-good (visited on 01/26/2026).

[30] *Nyquist–Shannon sampling theorem*. Wikipedia. URL: https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem (visited on 01/26/2026).